

Electronics and Computer Science
Faculty of Engineering and Physical Sciences
University of Southampton

Alexander White

2nd August 2023

MIMD Parallel Computing Design with Distributed
Memory using RISC-V Cores

Project supervisor: Iain McNally
Second examiner: Lie-Liang Yang

A project report submitted for the award of
BEng Electronic Engineering

Abstract

The aim of this project was to investigate the use of open-source RISC-V based CPU cores in a parallel computing network. High performance computing is difficult to achieve on a single processor, therefore parallel computing is essential to carry out computationally intensive tasks.

RISC-V is an open-source instruction set for CPUs that has seen an increase in popularity. There exist many open-source RISC-V processors available for use without restriction that are highly customizable and suitable to be used in a parallel computing network.

A 2D mesh network was built and configured to use the PicoRV32 and SCR1 RISC-V cores as well as the Arm Cortex-M0. The network has been tested and the performance of the network using these cores has been compared and evaluated.

Statement of Originality

I have read and understood the ECS Academic Integrity information and the University's Academic Integrity Guidance for Students.

I am aware that failure to act in accordance with the Regulations Governing Academic Integrity may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.

I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

I have acknowledged all sources, and identified any content taken from elsewhere.

I have used and adapted resources from Iain McNally's ARM System on Chip Example webpage [1] and I have adapted Jim Hamblen's "razzle" VGA driver, converted to SystemVerilog by Iain McNally. In addition, I have used the source files for the Yoosys PicoRV32 [2] and the Syntacore SCR1 [3].

I did all the work myself and have not helped anyone else.

The material in the report is genuine, and I have included all my data/code/design.

I have not submitted any part of this work for another assessment.

My work did not involve human participants, their cells or data, or animals.

Acknowledgements

I would like to thank my supervisor Iain McNally for his constructive feedback and technical advice throughout this project. I would also like to thank Lie-Liang Yang for his time examining this project.

Contents

Abstract.....	2
Statement of Originality.....	3
Acknowledgements.....	4
Contents	5
1 Introduction.....	7
2 Background.....	8
2.1 Parallel Computing	8
2.1.1 Memory Architecture.....	8
2.2 Network	8
2.2.1 Topologies	9
2.2.2 Routing.....	10
2.2.3 Buffering.....	12
2.3 Arm Cortex M0.....	13
2.4 AHB-Lite Bus	14
2.5 RISC-V	15
2.5.1 Picorv32	15
2.5.2 SCR1	16
2.6 Universal Asynchronous Receiver/Transmitter (UART)	16
2.7 Mandelbrot Set.....	17
2.8 Relevant Previous Work	17
2.8.1 Tait	17
3 System Design	18
3.1 Network	18
3.2 Processing Element.....	18
4 System Implementation	20
4.1 Router.....	20
4.1.1 Network Buffer	22
4.2 Processing Element.....	22
4.2.1 AHB-Lite Adapter	22
4.2.2 Performance Counter	23
4.2.3 Message Passer	23
4.2.4 Interrupts.....	24
4.3 Serial Link Node.....	24
4.3.1 Timing Module	24
4.3.2 Error Checking.....	25
4.3.3 Send/Receive State Machines	26
4.4 Bootloader Subsystem	27
4.5 Video Node	28
4.6 Packet Generator/Receiver	28
5 Testing	29
5.1 Testing Strategy	29
5.2 Network Testing	29
5.3 Serial Link Testing.....	29
6 Results and Analysis	30
6.1 Show Case Program.....	30
6.2 System Performance Results	31

6.2.1	PicoRV32 Variation Results	31
6.2.2	Core Comparison.....	33
6.2.3	Network Performance	37
6.3	Evaluation.....	38
7	Project Management.....	40
8	Conclusion.....	42
8.1	Future Work	42
9	References	43
10	Appendix A - Project Schedule	44
11	Appendix B – Risk Assessment	45
12	Appendix C – Serial Link Node Diagram	46
13	Appendix D – Video Node Diagram.....	47
14	Appendix E – Performance by PicoRV32 Variation	48
15	Appendix F – Results Data.....	50

1 Introduction

The purpose of this project was to develop a parallel computing network based on RISC-V CPU cores and to compare their performance relative to the Arm Cortex-M0. There exist many open-source RISC-V cores online that are free to use in a design without any restrictions [4]. The results of this project were used to investigate the feasibility and potential benefits of using RISC-V cores in the place of Arm cores for future projects.

The network was implemented on an Altera DE2-115 FPGA as a distributed memory computer organized in a 2D mesh network as shown in Figure 1. A network of processing elements (PEs) were connected together using a network on chip, communicating with each other via message passing. The output of any showcase programs were displayed onto a VGA display. The network was parametrized to use the Arm Cortex-M0 or a chosen RISC-V core and the system performance in a showcase program was measured and analysed. A serial communication link was used to output these performance results to a PC and also to rapidly reprogram the processing elements.

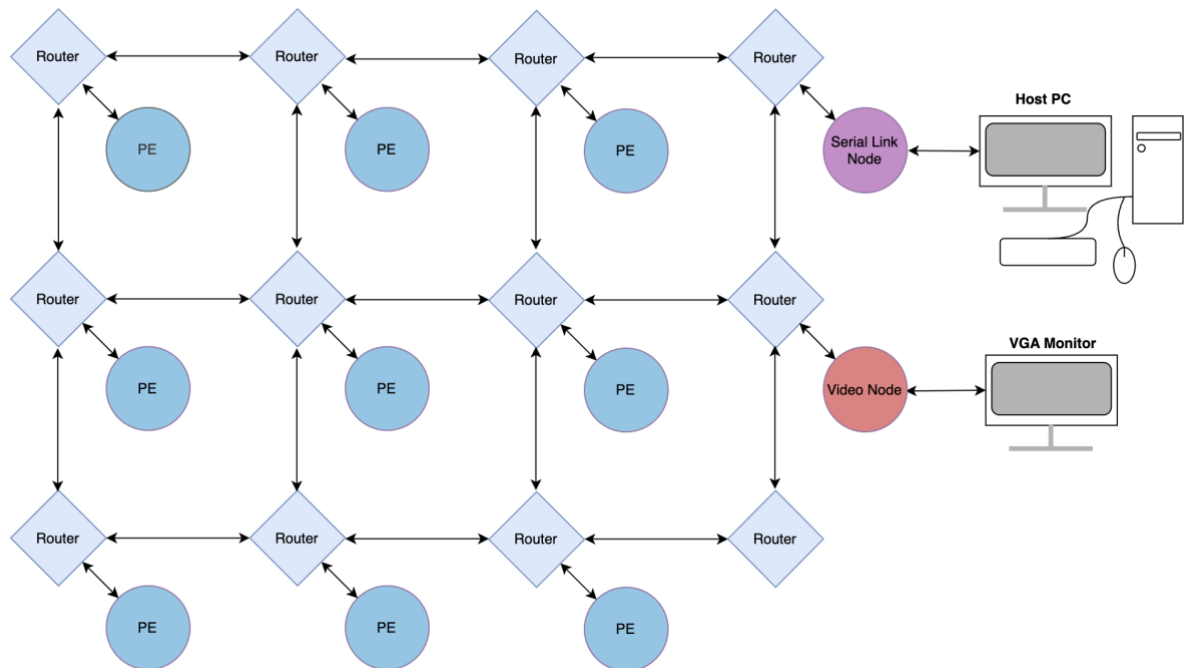


Figure 1 Parallel Computing Network

2 Background

2.1 Parallel Computing

MIMD computers use multiple instruction and data streams to execute individual tasks within a program in parallel to achieve a speedup in computation time, exploiting process level parallelism [5].

A MIMD computer is made up of an arbitrary number of processing elements communicating over a shared network. One processing element on the network can be assigned to manage the distribution of a shared task, to ensure that each processing element on the network is allocated a fair percentage of the workload.

2.1.1 Memory Architecture

Many modern computers use multicore processors to execute multiple instructions and data streams using a shared memory, connected as shown in Figure 2a. This approach to parallel computing is not suitable for a large number of processors as memory coherency problems will arise, requiring complex logic to achieve synchronisation [5].

An alternative method is to use a distributed memory system, whereby each processing element is allocated a local memory with a private address space as shown in Figure 2b. Processing elements can communicate with other processors by sending and receiving messages over a network, hence they are often called ‘Message Passing’ computers. Distributed memory systems are more predictable and require less complex circuitry to manage as each memory bank is managed by a single processing element [6].

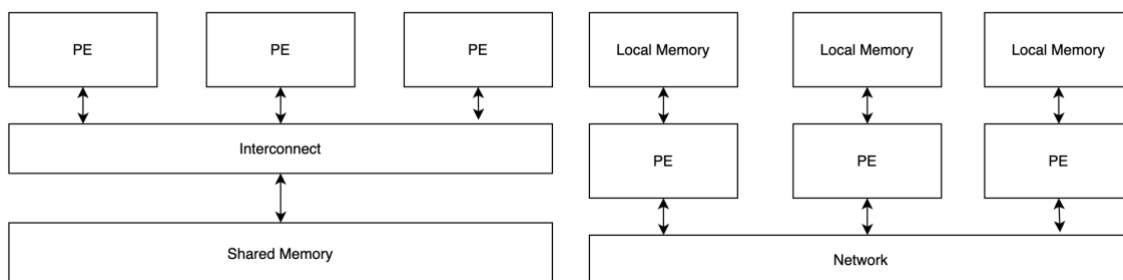


Figure 2 (a) Shared Memory Processor

(b) Distributed memory 'message passing' processor

2.2 Network

The design of the network over which processing elements communicate is an important factor in the performance, scalability, and reliability of the computing system [7].

Each processing element can communicate to other processing elements on the network using an attached router that will route messages between nodes. Messages are organized into packets containing flits (flow control digits), which are the smallest

amount of information that can be moved through the network. A diagram of a packet is shown in Figure 3. The first flit in a packet is a header which typically contains information such as the destination address of the packet and the number of flits within the packet.

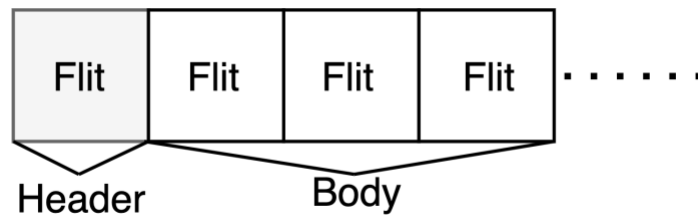


Figure 3

2.2.1 Topologies

The topology of the network is determined by factors such as the pattern of communication between nodes, area available on chip and the expected number of packets sent per cycle [7].

A fully connected network, where every node is connected to every other node, would provide the best performance, however, it would be expensive to scale. A simple topology, such as a linear array shown in Figure 4a, would be inexpensive to scale, requiring one channel per added node, yet would suffer from reduced bandwidth as the limited number of channels would be shared by multiple nodes. As a result, network topologies are designed to find a balance between cost and performance.

The performance of network topologies can be compared by calculating the greatest minimal routing distance between nodes on the network, referred to as the network diameter [7]. Another useful metric when comparing network topologies is a network's bisection bandwidth, which is used to help to identify bottlenecks which can be the cause of contention when the network is under load. Bisection bandwidth is calculated by finding the smallest summed bandwidth of channels connecting two halves of the network [1].

Figure 4 shows some common network topologies and Table 1 summarises their performance for N number of nodes:

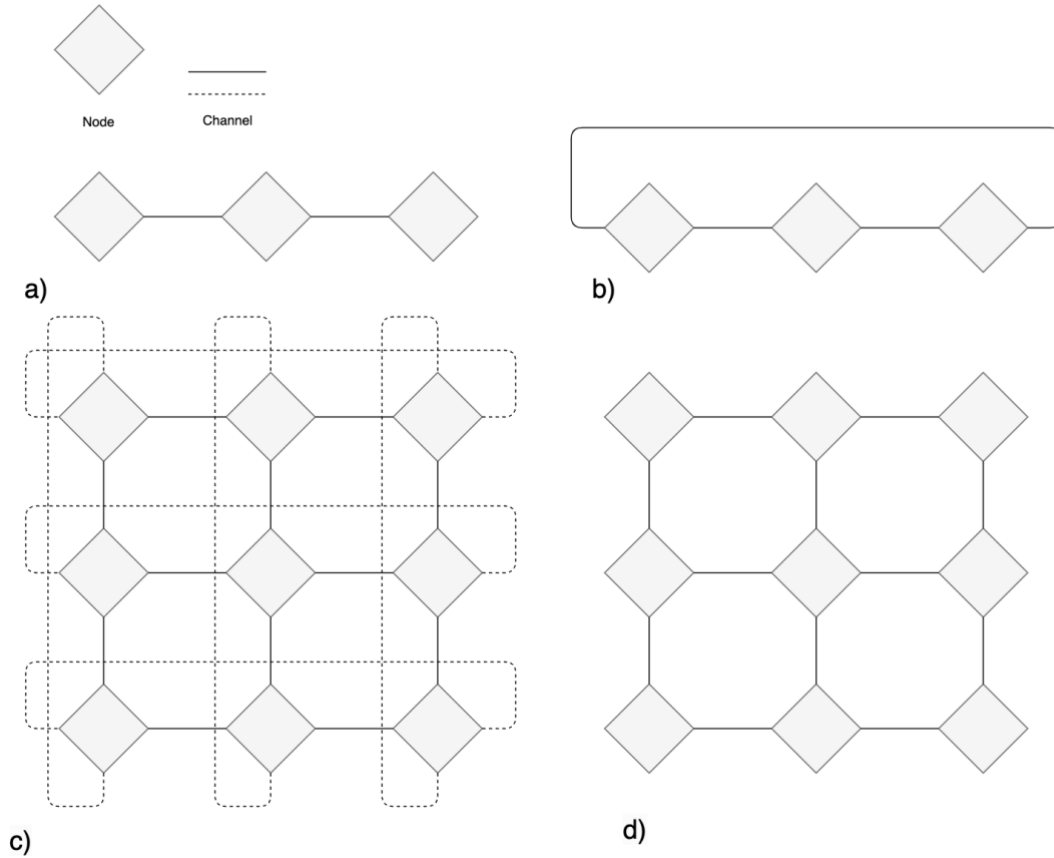


Figure 4 a) Linear Array b) Ring c) 2D Torus d) 2D Mesh

Table 1 Topology Comparison for N nodes with a channel bandwidth of B [6]

Topology	Network Diameter	Bisection Bandwidth
Linear Array	$N - 1$	B
Ring	$N/2$	$2B$
2D Mesh	$2(N^{1/2} - 1)$	$BN^{1/2}$
2D Torus	$N^{1/2}$	$2BN^{1/2}$

2.2.2 Routing

The routing algorithm for a network determines the path between non-adjacent nodes. Packets can be routed in a fixed path through the network or adaptively depending on the network state. When designing a routing algorithm, the aim is to reduce latency whilst using the minimum amount of logic possible. Care must be taken to prevent a circular dependency in routing, known as deadlock [7].

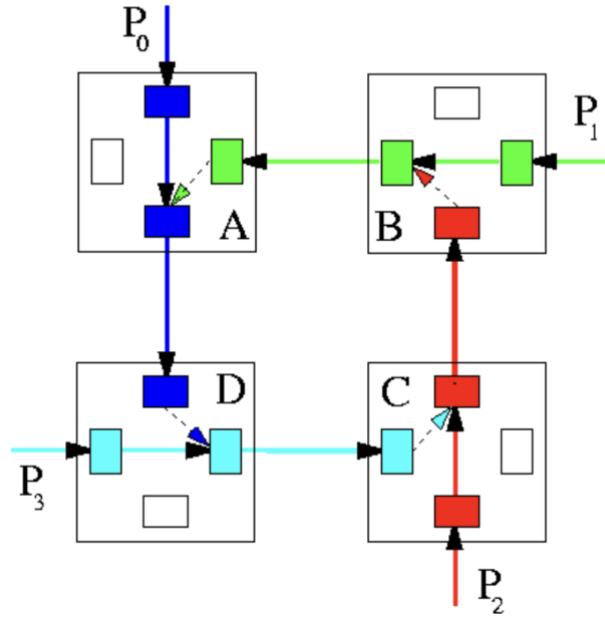


Figure 5 Deadlock Diagram reproduced from [8]

Deadlock is caused when two or more packets on the network cannot progress to the next router as they are circularly waiting for a shared buffer to be available [8]. Figure 5 demonstrates a typical example of deadlock in a network that uses one buffer slot per input channel. In the diagram, each packet is represented by a distinct colour and is made up of multiple flits occupying buffer spaces. In this case, each packet is attempting to turn left but they are each blocked by other packets. There is a circular dependency as P0 is blocked by P3 which is in turn blocked by P2, and so on until P0 is blocked by itself.

2.2.2.1 Dimension Ordered Routing

An example of a fixed routing algorithm is dimension ordered routing. In this routing scheme, each dimension must be travelled in sequence, therefore in XY dimension ordered routing, a packet will be sent in the X direction until it reaches the target X co-ordinate then in the Y direction until it reaches the final co-ordinate of the target node [9]. In this routing scheme, each dimension of travel is a separate virtual network within the network, using separate buffer spaces. This results in legal turns shown in Figure 6, preventing a circular wait dependency as previously demonstrated in Figure 5.

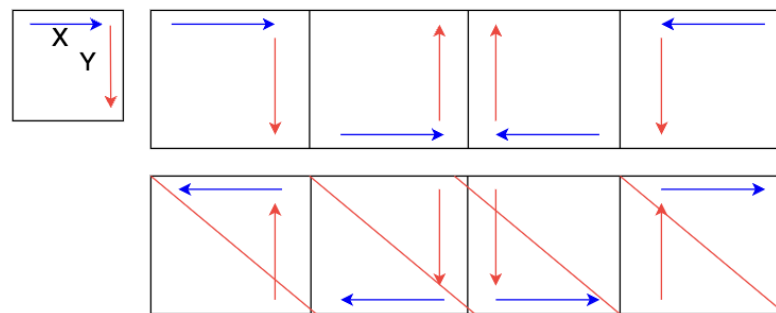


Figure 6 (a) Legal and (b) Illegal turns in XY dimension order routing

2.2.3 Buffering

The network uses First In, First Out (FIFO) buffers to temporarily store data to manage the flow of data. A FIFO buffer design typically uses a write pointer that initially points to location zero in the buffer and increments every time data is written to the buffer. The write pointer will wrap around to location zero when it increases past the maximum address.

Similarly, a read pointer points to the location of the next unit of data to be read out of the buffer, a location sometimes referred to as the ‘head’ of the buffer. Figure 7 shows a FIFO with read and write pointers large enough to address the entire buffer address space. In this configuration, a full state is calculated when the write and read pointers point to the same address. Unfortunately, this FIFO design can also be empty when the write and read pointer point to the same address, as shown in Figure 7.

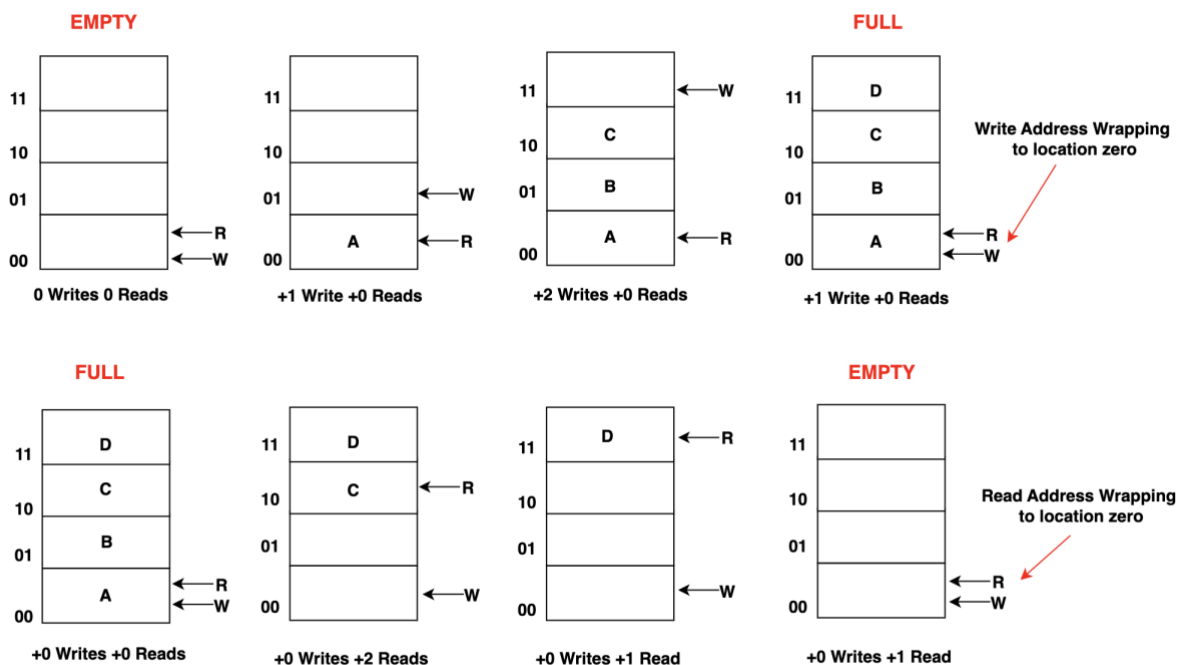


Figure 7

Figure 8 shows a new FIFO buffer configuration with an address size of four, that allocates an extra bit to the read and write pointers, creating a ‘virtual’ address size of eight. The extra bit can be interpreted as tracking if the pointers have wrapped around to zero in the original buffer address space.

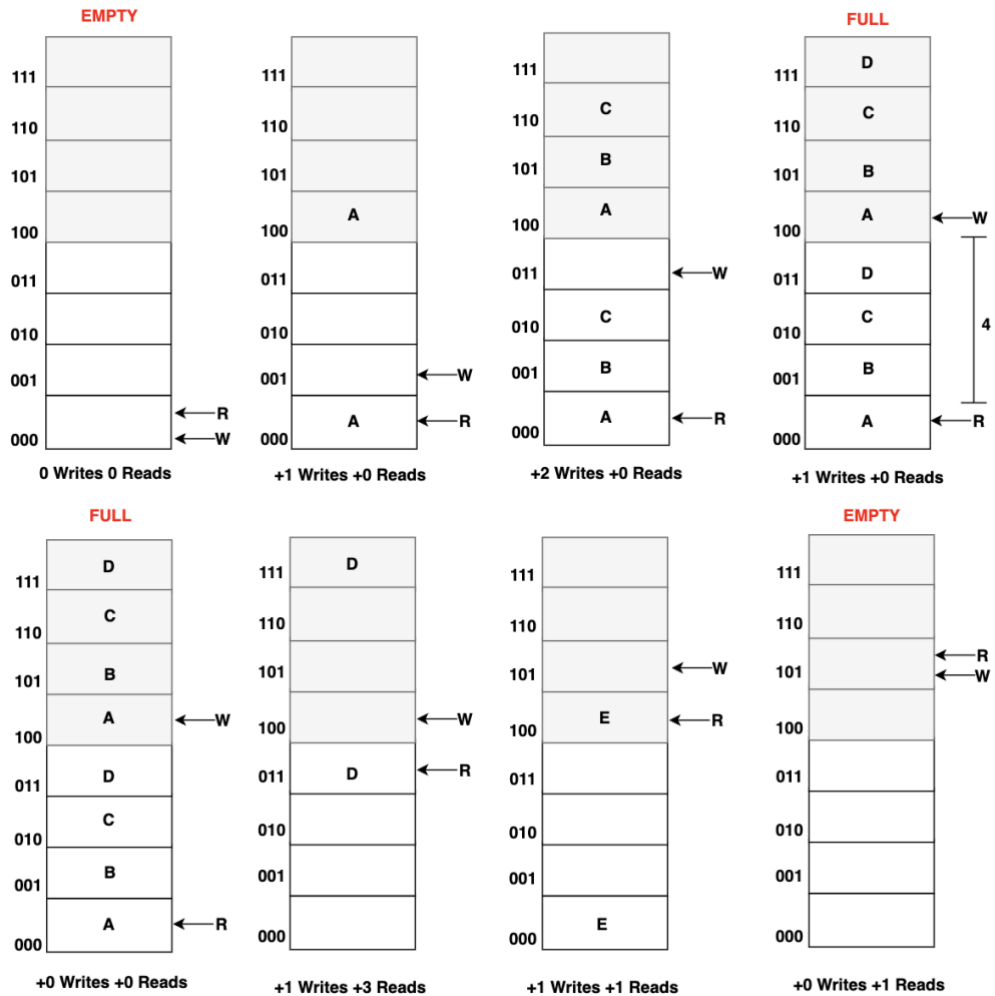


Figure 8

The diagram shows that when the buffer is empty, the values of the pointers are the same, as they have wrapped around the address space the same number of times and point to the same address. The buffer is full when the pointers have wrapped around the address space a different number of times and are pointing to the same address.

The empty flag will be set when the values of the read and write pointers are the same. The full condition can be detected simply by checking if the MSB of the read and write pointers are different and the remaining bits are the same [10].

2.3 Arm Cortex M0

The Arm Cortex M0 is a small 32-bit core designed for embedded applications that can support interrupts and dedicated multiply instructions [11]. The Cortex-M0 is available from Arm for educational purposes and is used widely for both university projects and commercial applications. However, the source RTL for the Cortex-M0 is obfuscated, meaning it is unreadable yet synthesisable, preventing designers from making changes to the core to suit their requirements.

2.4 AHB-Lite Bus

The AMBA Advanced High Performance Lite (AHB-Lite) bus is a System on Chip bus protocol developed by Arm [12]. It is used by the Cortex-M0 to communicate with memory and peripherals. Using the AHB-Lite terminology, the M0 is referred to as a ‘Master’ and the memory and peripherals accessed via the bus are known as ‘Slaves’.

Figure 9 shows an AHB-Lite bus master read a location in memory using the AHB-Lite bus protocol. The transaction is split up into address and data phases over two clock cycles. In the address phase, the master requests data from the AHB-Lite slave by asserting the address on the HADDR wire. On the next clock cycle, the data phase begins as the slave responds by asserting the HREADY signal and asserts the data at the given address on the HRDATA wire.

As the address and data phases are in separate clock cycles, the address phase of the next transfer can be asserted whilst the data from the last cycle is being returned. This is referred to as pipelined memory access and can be used to increase the throughput of the bus.

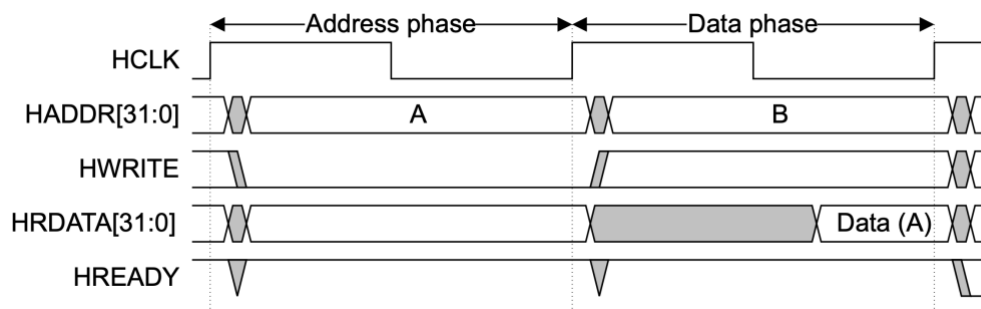


Figure 9

System on chip peripherals, such as timers and communication interfaces, can also be accessed in this way. They are referred to as memory mapped peripherals as they act as if they are an address in memory from the perspective of the programmer. To access these peripherals, the programmer simply writes or reads to the address assigned to that peripheral. A bus interconnect network can be used to decode the address asserted on HADDR to select a certain peripheral connected to the bus, shown in Figure 10.

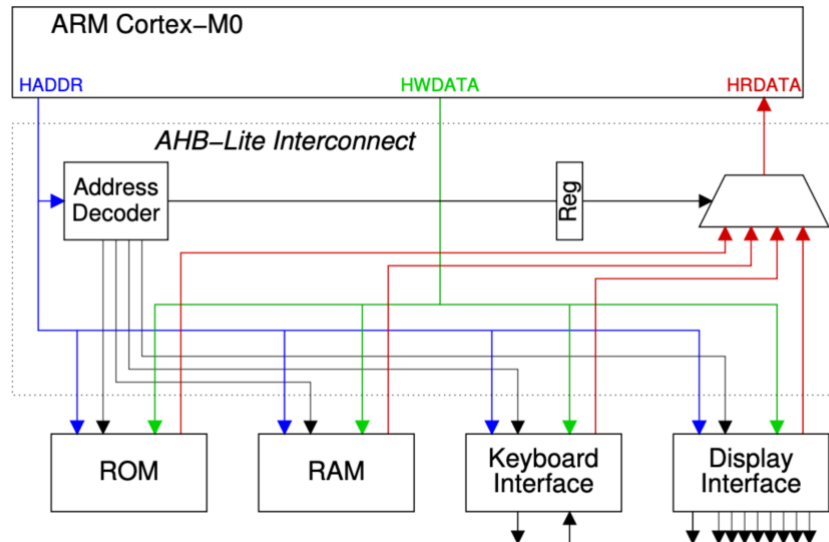


Figure 10 Diagram of AHB-Lite interconnection between the Arm Cortex-M0 and example peripherals, reproduced from [13]

2.5 RISC-V

RISC-V is an open-source instruction set for CPUs developed in 2010 currently run by the RISC-V Foundation, a non-profit organization [4]. An instruction set defines the functionality of the CPU and is the interface between hardware and software. The RISC-V instruction set can be used without a license, allowing companies or students to easily develop processors.

The base RISC-V 32-bit instruction set is capable of being extended to include features such as integer multiplication, division or compressed instructions. Compressed instructions are shorter instructions that only use 16-bits of memory and can be used to reduce the size of program memory and the number of memory reads. The RISC-V specification states that typically 50% of all instructions can be compiled as compressed instructions, leading to a 25% reduction in memory required to store a typical program [14]. A 32-bit RISC-V core supporting compressed instructions and multiply and divide instructions can be referred to as an RV32IMC core, with RV32I referring to the base 32-bit instruction set, M referring to the multiply and divide extensions, and C referring to support for compressed instructions.

A RISC-V processor implementing the RV32IM instruction set is most comparable to the ARM Cortex M0. There exist many open-source implementations of the RV32IM instruction set, designed for different applications. The absence of a license fee and flexibility of RISC-V cores make them an attractive option for use as a processing element within a MIMD computer.

2.5.1 Picorv32

The PicoRV32 is a 32-bit open-source RISC-V core supporting the RV32IM instruction set, developed by Yosys [2]. It is designed to be a small configurable CPU to be used as an auxiliary processor for a larger system design on an FPGA. The processor is

distributed under the ISC licence, allowing anyone to use and customize the core for any application.

Optional features can be enabled on the core such as a barrel shifter, dual port registers or compressed instructions. Some features are not needed for a processing element within a parallel computer, and therefore can be removed to save resources on the FPGA.

2.5.2 SCR1

The SCR1 is another free to use RISC-V core supporting the RV32IM instruction set developed by Syntacore [3]. The core’s source code is freely available and written in System-Verilog. Like the Cortex M0, the chip has been implemented in silicon and has extensive documentation. Similar to the PicoRV32, the core is optimized for power and area and has many parameters that can be changed for a specific application.

2.6 Universal Asynchronous Receiver/Transmitter (UART)

Universal Asynchronous Receive Transmit (UART) is a serial protocol used to transfer data between devices without a shared clock [15]. The link only requires transmit and receive wires but Clear to Send (CTS) and Request to Send (RTS) wires may be used for flow control.

A UART connection is made up of frames that typically transfer eight bits of data at a time. Before a UART frame begins, the signal is kept in a high idle state and when the frame starts, a start bit is signalled by setting the signal low for one period.

The data bits are then sent serially at a bit rate previously agreed by the two devices. The receiver may use a technique called ‘oversampling’, whereby each data bit is sampled multiple times and the average bit value is used in order to exclude noise in the signal. A parity bit can either be set as a one or a zero to ensure that the total number of high bits is even or odd, enabling the receiving device to detect if there has been a single bit error.

Figure 11 illustrates a UART frame using a parity bit along the transmit wire ‘UART_TXD’.

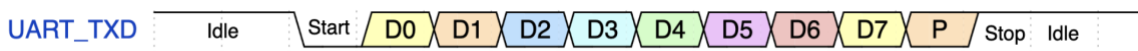


Figure 11

2.7 Mandelbrot Set

The Mandelbrot Set is a mathematical set of numbers that can be visualised to create a fractal pattern. It is defined as the set of complex numbers, C_n , that do not cause Z to go to infinity when iterated using Equation 1 [16].

$$z_{n+1} = z_n^2 + C \quad z_0 = 0 \quad (1)$$

A colourful fractal pattern can be generated by assigning a colour to each co-ordinate on the complex plane corresponding to the number iterations taken to exceed the threshold value by that co-ordinate.

The Mandelbrot set calculation is an example of a computationally intensive task that is ‘embarrassingly parallel’. This means that it can be calculated in parallel by an arbitrary number of independent processing elements using little communication.

A program to calculate the Mandelbrot set may be used to test the functionality of a Network on Chip whilst producing a visual result.

2.8 Relevant Previous Work

Previous student projects have developed parallel computing networks using the Arm Cortex-M0 processor. Lam designed a distributed memory, parallel computer that was spread across two FPGAs connected using a 50MHz link [17]. Siabi was able to run a parallelised program using two FPGAs each with 8 processing elements [18]. Woolmer extended the network to use an extra dimension, creating a 4D hypercube network across two FPGAs using dimension ordered routing [19].

2.8.1 Tait

In 2018, Tait developed a virtual memory system that uses the hardware of a distributed memory computer yet uses a shared address space via communicating memory access requests [20]. The memory architecture only showed a minor speed up to complete a showcase program compared to a message passing system, however the new system was able to use a simpler showcase program and allowed non-parallel programs to be ported with less modification.

3 System Design

The proposed design is based upon a Processing Element (PE) communicating with other processing elements via a 2D mesh network. Routers route packets sent from nodes on the network using XY dimension order routing. A Serial Link node enables a PC to rapidly reprogram the memory of the processing elements and allows processing elements to output quantitative performance results. A bootloader system enables the instruction memory of each processing element to be reprogrammed after the network has been synthesised. Processing elements can write to an external display by sending packets to a video node on the network.

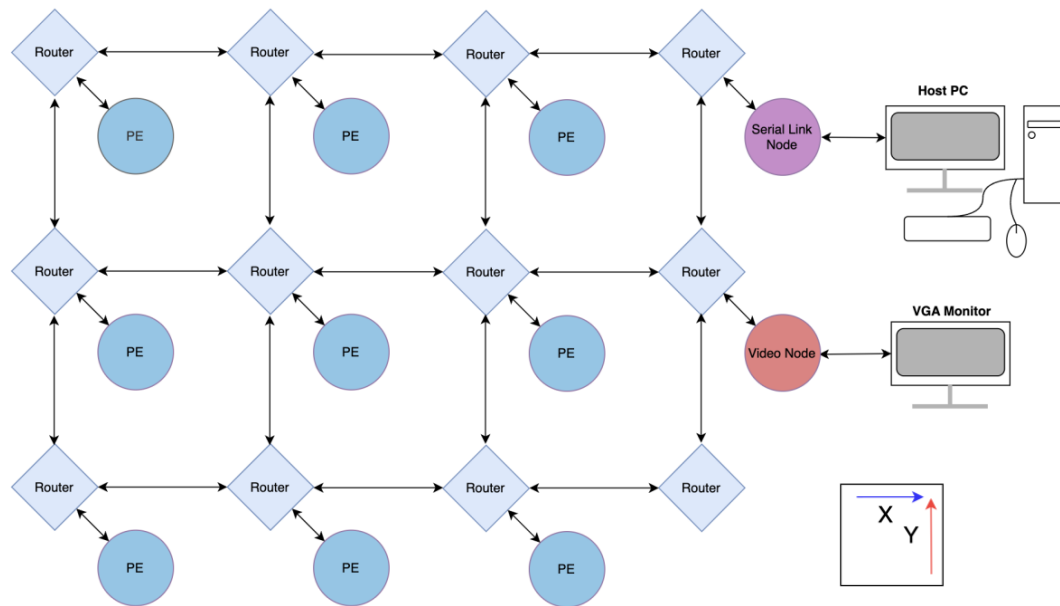


Figure 12 2D Grid Network of processing and routing elements

3.1 Network

The 2D mesh network topology has a bisection bandwidth of three times the individual bandwidth of each channel and has a network diameter of five. The network routers use XY dimension ordered routing to avoid deadlock. A router in the South-Eastern corner of the network is required when using XY dimension ordered routing to allow processing elements on the bottom row to communicate with the video and Serial Link nodes.

3.2 Processing Element

The processing element is based on a CPU core that communicates with memory and peripherals by using a 32-bit AHB-Lite bus. A local RAM module stores the instructions

and data of the program and a message passer allows the processing element to send packets onto the network. The message passer is connected to the network by 32-bit wide PE IN/OUT signals that can carry a single flit per clock cycle each. A full signal is used to pause the flow of flits and a send signal is used to indicate that a device is ready to send flits.

A performance counter module is used to gather performance information such as the program cycle count and is able to be read from and reset by the core.

A Bootloader module is also an AHB-Lite master on the bus that can be enabled via the ‘Bootloader Enable’ signal that is activated during the bootloader process described later in section 4.4. When activated, the bootloader will write packets received from the network to memory and the CPU core will be disabled.

Figure 13 shows a block diagram of a processing element, showing the bus masters above the AHB-Lite interconnect and the bus slaves below.

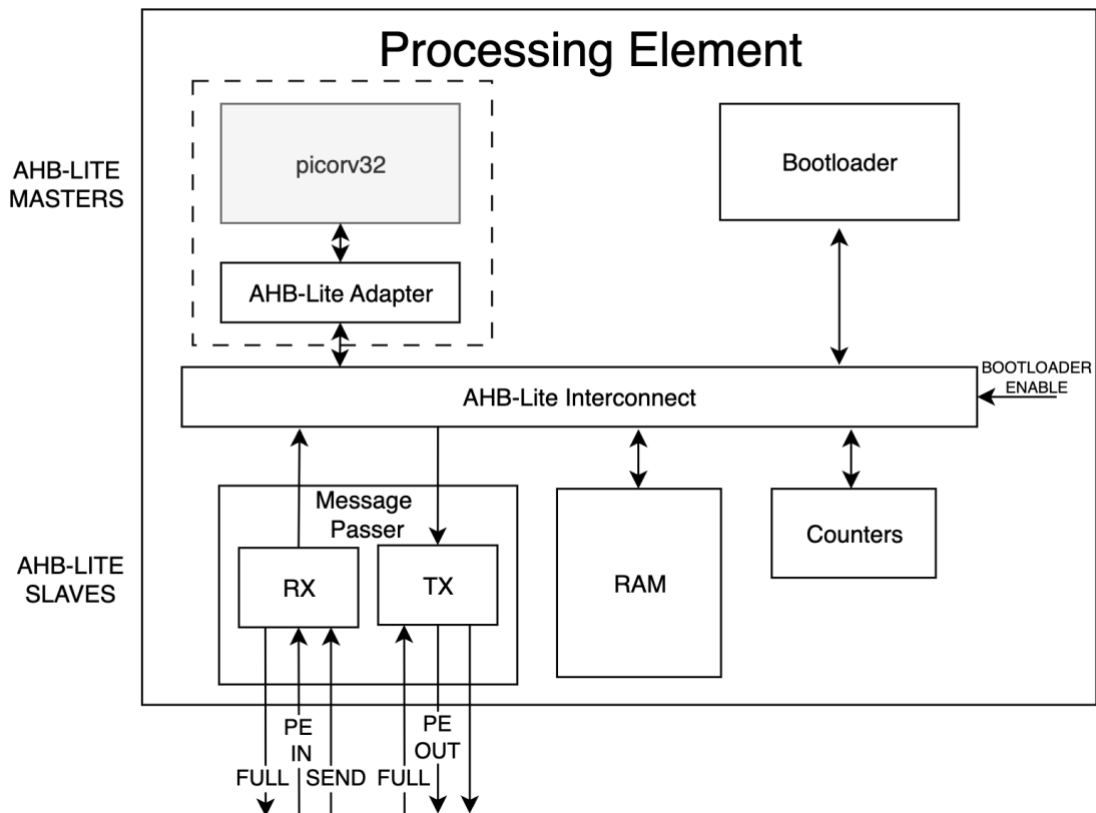


Figure 13

All processing elements are configured identically, apart from the core and bus adapter within the dashed line shown in Figure 13, which can be switched out to use different a core. The network can be configured to change parameters such as the core and the size of the RAM for each processing element. The AHB-Lite adapter module bridges the PicoRV32 native memory interface to the AHB-Lite bus.

4 System Implementation

4.1 Router

The network router uses multiplexers to route packets between nodes in the network according to dimension ordered routing. A router is comprised of ‘network buffers’ with the “RECEIVE” ports of the buffers connected to input channels from adjacent routing elements as shown in Figure 14. The network buffers receive request and data signals from incoming channels, each asserting a full signal on their input channels when the buffer full condition is met using the ports described in section 4.1.3. Priority is given to packets travelling in Y-Axis over the X-Axis as the priority of the router is to empty the packets on the network. This means that the packets sent from a processing element onto the network always have the lowest priority.

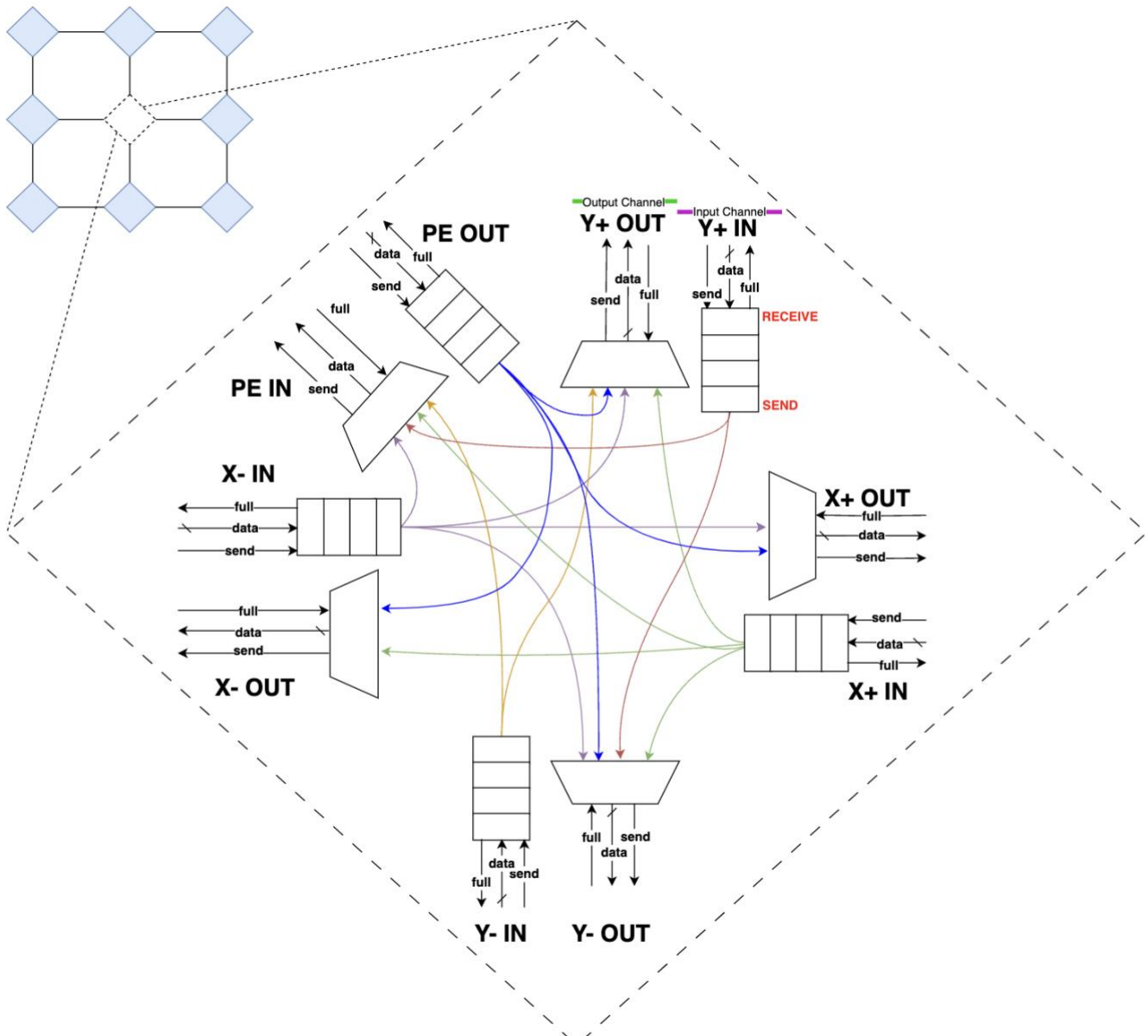


Figure 14 Routing Element

The “SEND” port of a network buffer is connected to all the possible output channels according to XY dimension ordered routing. Each line colour represents a different network buffer (e.g Red - Y Plus network buffer) and each line contains the packet pending, request, destination outputs and the blocked input to the network buffer as described in section 4.1.3. This means that if a flit is present within a network buffer, a request signal is asserted to all its output channel multiplexors.

Dimension order routing ensures that packets in the Y-axis can only be routed along the Y-axis or into the local processing element, simplifying the multiplexing logic along the X-axis. This can be seen as the “X Plus Out” direction multiplexor only has two input channels, PE OUT and X Minus In.

The block diagram for the Processing Element IN (PE IN) channel multiplexor is shown in Figure 15. The logic within the multiplexer module will determine if the packet needs to be routed in the direction of its channel by comparing the requested destination to the node ID. If the multiplexor is not blocked, not routing another packet, and the destination of the input packet is evaluated to be in the direction of the channel, the multiplexor will allow the input buffer to send flits into the channel until the entire packet is routed to the next buffer. In the case that there are requests from multiple input channels, the input channel with the highest priority will be routed to the next buffer.

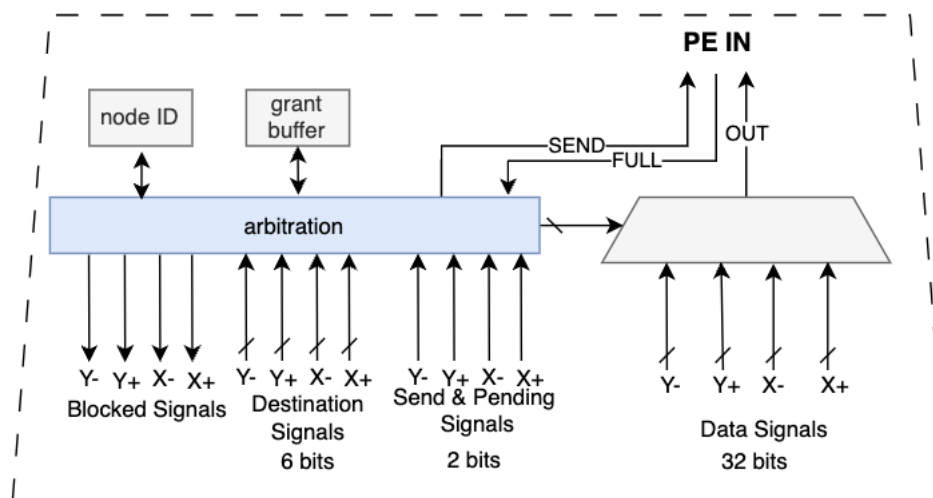


Figure 15 Multiplexing Logic

The “grant buffer” stores the direction from which the packet is travelling from, so that the channel is still reserved for that direction after the header flit has been routed and the destination signal de-asserted. The receiving buffer in the next routing element asserts a full signal if it is full, causing a blocked signal to propagate back to the network buffer sending the packet. When the blocked signal is de-asserted, the flits will continue to flow to the next buffer.

Due to the cycle delay between packets sent by a network buffer, described in section 4.1.1, if there is a packet following from the same direction immediately after the last routed packet, its network buffer will not generate a request until a cycle later. During this delay, a packet from a lower priority direction can reserve the channel and be routed to the next buffer. This avoids a scenario where a high traffic routing element only accepts data from a high priority channel. For example, a processing element in the Northwest corner of the network would only accept data from the negative Y direction when the network is at full capacity without this feature.

4.1.1 Network Buffer

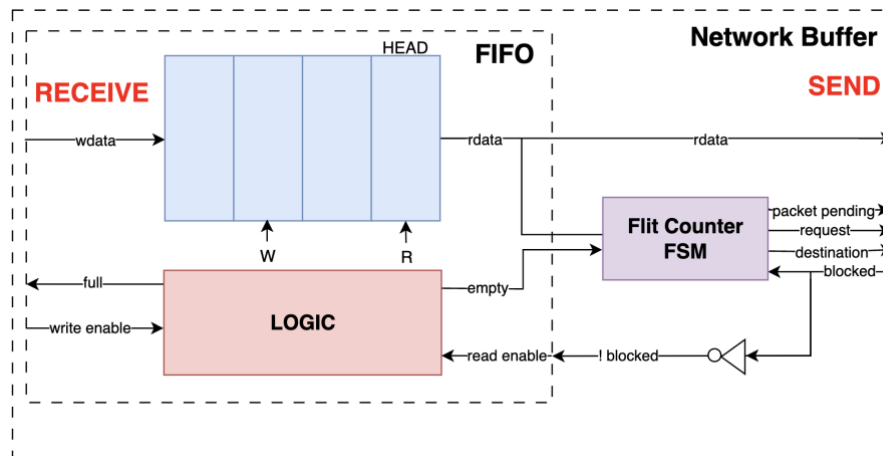


Figure 16 Network Buffer Diagram

The Network Buffer is an example of a Virtual Cut Through buffer that allows flits to immediately be sent to the next buffer after they are received. It uses a FIFO to store flits when a blockage occurs.

Input flits are received from the “RECEIVE” port and flits are transmitted from the “SEND” port of the network buffer. The module decodes the header flit of an incoming packet and stores the destination of the packet and the number of flits within the packet to a register. A Finite State Machine (FSM) detects when a new packet is at the ‘head’ of the buffer. When this is detected, the destination of the packet is asserted along with a request signal, communicating that a new packet is ready to be sent. A packet pending signal is asserted when a body flit of a packet is at the head of the buffer as the packet has not been fully sent. There is a one cycle delay between successive packets in the buffer which is useful in resolving network contention.

4.2 Processing Element

4.2.1 AHB-Lite Adapter

An AHB-Lite adapter connects the native memory interface of the PicoRV32 core to the AHB-Lite slaves. The AHB-Lite adapter supports the lookahead extension on the PicoRV32 core to speed up read times by accessing the next read address one cycle earlier. This signal can also be used to provide the address of a read or write request one

cycle before the core expected the data to be asserted from the slave. This allows the native interface to be mapped to address and data cycles of the AHB-Lite protocol.

Figure 17 shows in the red circles the PicoRV32 memory interface signals that can be mapped to their corresponding AHB-Lite signals in the address phase. Similarly, the green circles show the signals that can be mapped to the data phase of a read transaction.

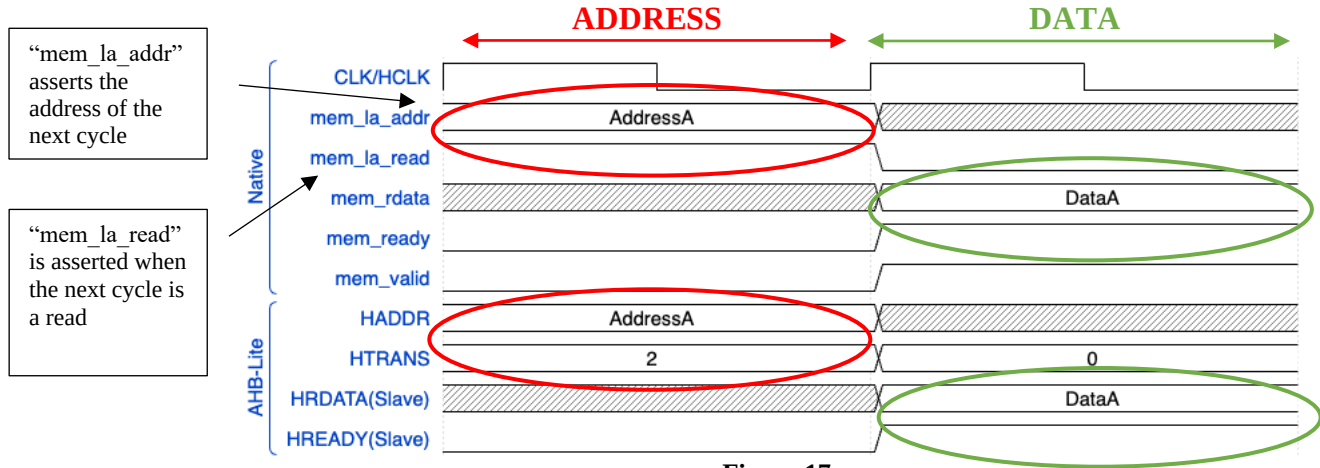


Figure 17

Unfortunately, as the PicoRV32 only supports one outstanding memory read or write at once, therefore it is not possible to run two concurrent memory transactions at a time.

4.2.2 Performance Counter

The Performance Counter module is connected to the AHB-Lite bus and can therefore monitor the address and ready signals to detect how many times the processing element has read or written to memory or certain peripherals. The collected data can then be sent over the serial link to the host PC.

4.2.3 Message Passer

The message passer is an AHB-Lite slave that is the interface between the processing element and the network. The message passer receives individual flits of a packet to be sent from the core over the AHB-Lite bus with a varying number of cycles between flits. To ensure flits within a packet are sent in consecutive cycles, the message passer buffers the flits in the “TX BUFFER”, then sends them in consecutive cycles when the entire packet is buffered. A bit will be written to the status register when the TX buffer is full to prevent the CPU core from attempting to write to the message passer.

Similarly, an “RX BUFFER” receives packets from the network and is large enough to store the maximum size packet, guaranteeing that when the message passer is ready to receive a packet, it can receive flits in consecutive clock cycles, preventing blockages in the network.

The module sets a bit in its status register when a flit is buffered in the RX buffer. The master device can read the status register to poll if a flit has been received before reading a flit from the buffer. The message passer can also send an interrupt to the CPU when a

message has been received, freeing the processing element from constantly polling the status register until a message has been received.

4.2.4 Interrupts

An interrupt from the message passer indicating a message has been received triggers an interrupt handler in the CPU core that will move messages from the message passer RX buffer onto a circular buffer implemented in software. This empties the message passer and allows it to receive new messages and clear the network. If the message queue becomes full, the interrupt will be disabled, allowing the CPU core to service queued packets.

4.3 Serial Link Node

Any processing element on the network can communicate with the host PC by sending a packet to the Serial Link node, which will send the body of the packet to the PC. The PC can send packets to any node on the network by sending an entire packet including the header to the Serial Link node. The Serial Link node is connected to a router using the same PE IN and PE OUT signals as the processing element.

When transmitting, the node buffers data from the network and converts packets into serial frames and performs the reverse when receiving packets. The DE2-155 development board includes a ZT3232 transceiver which allows the serial data to be sent using the RS-232 standard using a 9-pin connector to a PC. A detailed diagram of the system is illustrated in Appendix C.

4.3.1 Timing Module

Each bit within a serial frame is sampled 16 times and each of the samples are biased to the middle of a bit period. An average bit level is calculated from the samples to determine the most likely bit level.

The timing module is used to generate the timings to sample the incoming serial data frame and to generate a signal when the next bit in a frame begins. Figure 18 illustrates the signals generated by the timing module set to oversample rate of four next to the received serial frame.

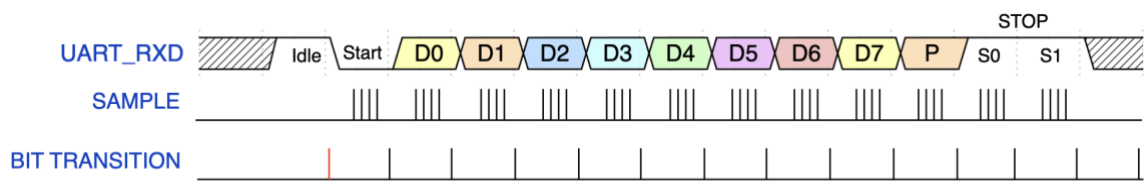


Figure 18 UART Receiver Sampling Timing

When transmitting, the timing module triggers the next bit of data to be sent on the TXD line (transmit line).

4.3.2 Error Checking

The parity bit is calculated in the serial transmitter by taking the XOR of zero and the first bit, then taking the XOR of the result with successive bits until the frame is finished, using the circuit shown in Figure 19. The parity bit is set to the result of this operation to ensure that there are an even number of high bits in the frame.

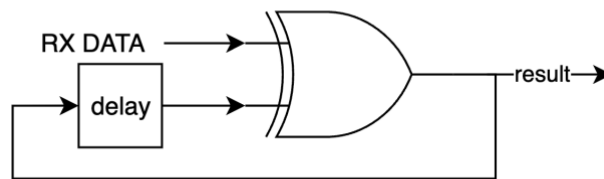


Figure 19

The serial receiver uses the same process as the transmitter, taking the XOR of all data bits and the parity bit to detect if an even number of high bits has been received. If an odd number of bits have been received, the receiver enters an error state and then de-asserts the clear to send signal.

The received data will be used to program the instruction memory of the processing elements; therefore, it is important that a corrupted frame is not able to be converted into a packet.

4.3.3 Send/Receive State Machines

The state machine of the serial receiver is illustrated in Figure 20.

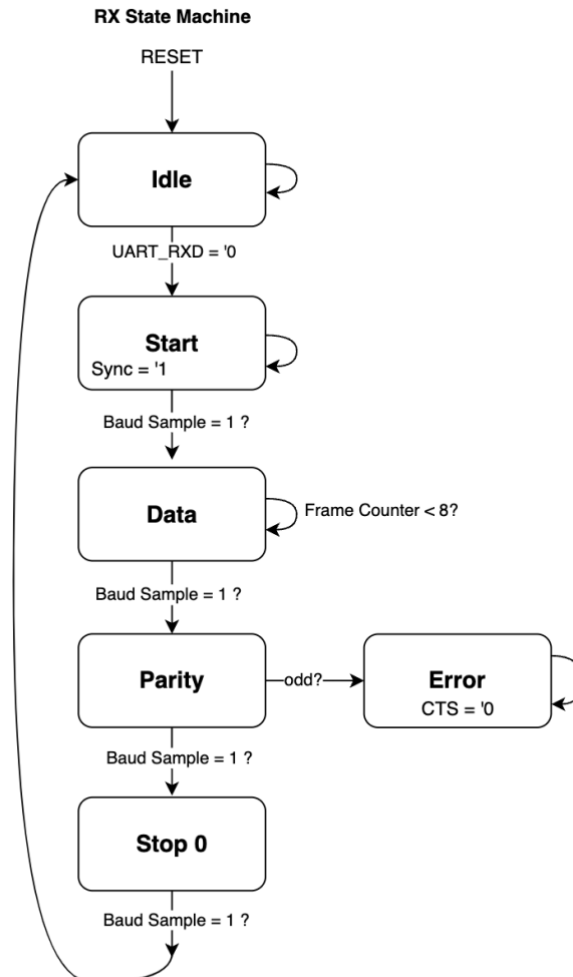


Figure 20

When the receiver is in Idle state, if a negative edge on the receive wire (`UART_RX`) is detected, the state machine goes into the Start state. After the signal is sampled, if the receiver determines that it has received a low bit, the state machine will progress to the Data state, if not, the receiver will go back to the Idle state. This prevents noise from initiating a frame in error.

4.4 Bootloader Subsystem

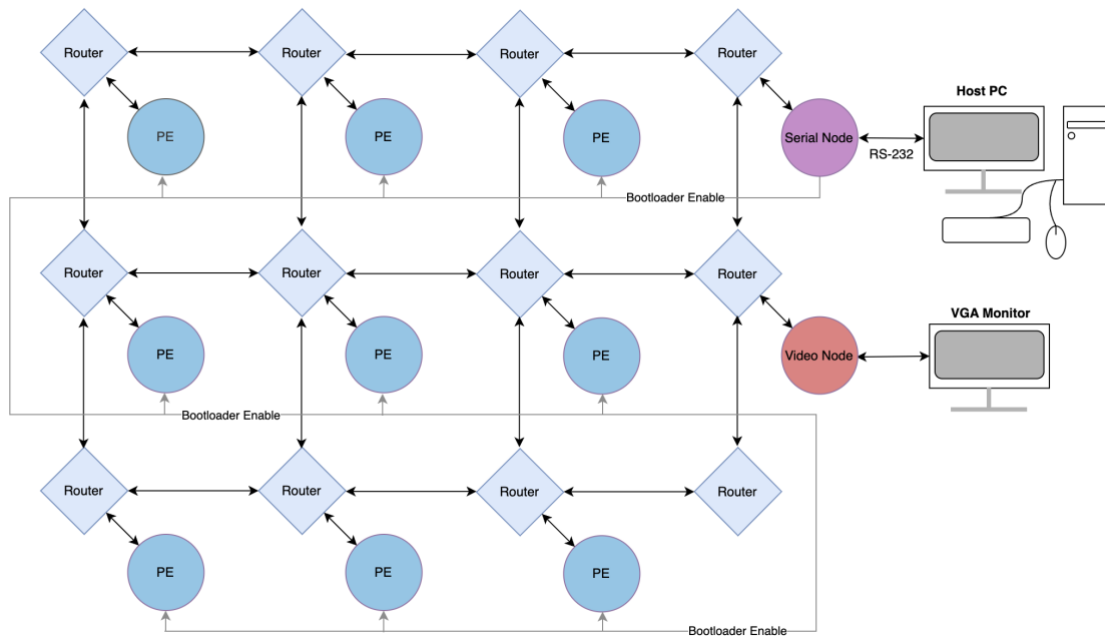


Figure 21

A bootloader subsystem enables the instruction memory of each PE to be reprogrammed after the network has been synthesised. A Python script was written to generate the packets from a .hex program file that could be sent over the network and also to co-ordinate the bootloader sequence.

The bootloader sequence is started by the host PC sending an 'echo' command to all processing elements in the network to confirm the number of processing elements on the network as well as the instruction set of each.

Next, a 'Start Bootloader' command is sent to the network from the host PC, which is then decoded by the Serial Link node. This causes a global reset signal to be sent to the entire network which empties buffered flits in the network. The bootloader enable is then asserted to set the bootloader module to be set as the master for all processing elements. The bootloader enable signal can be seen in Figure 21. This signal also disables all processing elements that are not being programmed from communicating on the network during the bootloading process, which could potentially corrupt the instruction memory of other processing elements.

The host PC subsequently sends packets containing the destination of the target processing element and the instruction words of the new program to individual processing elements through the Serial Link node and existing network channels. The bootloader module reads messages from the message passer and writes them to instruction memory.

This design allows the host PC to act as if it were a processing element on the network, using the same packet protocol to communicate. It also allows the PC to program an arbitrary processing element individually with separate programs if required.

Once all the instructions have been sent by the host PC, an ‘End Bootloader’ command signal is then sent, causing the global bootloader enable signal to be de-asserted and the network to be reset. This allows the CPU to take over from the bootloader and start from instruction 0x00 in memory. Using the same circuitry, the host PC can issue a network reset arbitrability, a useful feature for running test programs automatically.

4.5 Video Node

The video node interfaces to the network the same way as the Serial Link element described in Section 4.3. The detailed subsystem diagram of the video node is illustrated in Appendix D. A VGA driver adapted from [21], writes pixel colour data stored in the framebuffer to a 640 by 480 pixel monitor.

When a processing element writes a pixel on the display, it sends a packet to the network address of the video element. The packet contains a header and single data flit, with the data flit encoding the X and Y co-ordinates of the pixel and an associated eight-bit colour value.

When packets are received by the network buffer, the header is discarded, and the address is decoded to a format that allows the pixels within the frame to be stored consecutively in memory. A dual-port block ram instance is used as the framebuffer and allows pixels to be written to it at the same time that the display counter reads pixels from it.

In this implementation, only the four most significant bits of the colour value are used as the framebuffer is limited to store only four bits per pixel by the memory resources on the FPGA.

4.6 Packet Generator/Receiver

To test the network in simulation, a packet generator/receiver node was designed. Each core in the network can be replaced with a packet generator/receiver by selecting a parameter at the top-level module for the network. A Python script generates packets of random lengths and destinations, containing a string of random hex values. Each packet is sent by the packet generator at a configurable packet injection rate set as a parameter. Received packets are unloaded from the network at a constant rate of 1 flit per cycle by the receiver and then written to a file.

5 Testing

5.1 Testing Strategy

During implementation, each module was tested using basic testbenches and stimuli. Further simulations were conducted by writing C code to be run on cores that stimulated certain submodules.

The video node subsystem was tested mainly on hardware as the time taken to simulate the system writing a small object to the screen would take a prohibitively long time.

5.2 Network Testing

A major concern was the possibility of deadlock within the network therefore the packet generator was used to load the network with the intention of detecting this.

Approximately 45,000 packets were sent in total by the packet generators during simulation to test if the network would become stalled by deadlock. The network did not suffer from deadlock and the packets continued to be sent and received despite the increase in network traffic corresponding to a packet injection rate of 0.01 packets per cycle. This test does not prove that the network is deadlock free, however it suggests that this is unlikely.

5.3 Serial Link Testing

The serial link module was thoroughly tested by programming a processing element to receive packets from the host PC and then send the inverse of the data back to the PC. A Python script generated random numbers, organised them into packets of varying size, then sent them to a processing element within the network. A shared c .dll file was used so that the processing element and the PC could use the same C program to calculate the inverse function and match the inverse packets from the serial link. Approximately 400,000 serial frames were sent without a single error detected by the program.

6 Results and Analysis

6.1 Show Case Program

The Mandelbrot Set was calculated to demonstrate the functionality of the network and compare the performance of the cores.

Each pixel on the screen was converted to an imaginary number, with the centre of the screen set as $0 + 0j$, then the escape time equation (1) was iterated 255 times for each pixel. If the value did not iterate to a magnitude greater than two, it was assumed to be within the Mandelbrot set and the corresponding co-ordinate on the screen was assigned the highest possible colour value (white). If the magnitude exceeded two during this calculation, the number of iterations taken to do so was encoded to a colour and was written to the associated pixel.

Figure 22 shows the output of the showcase program on the screen when using eight-bit colour.

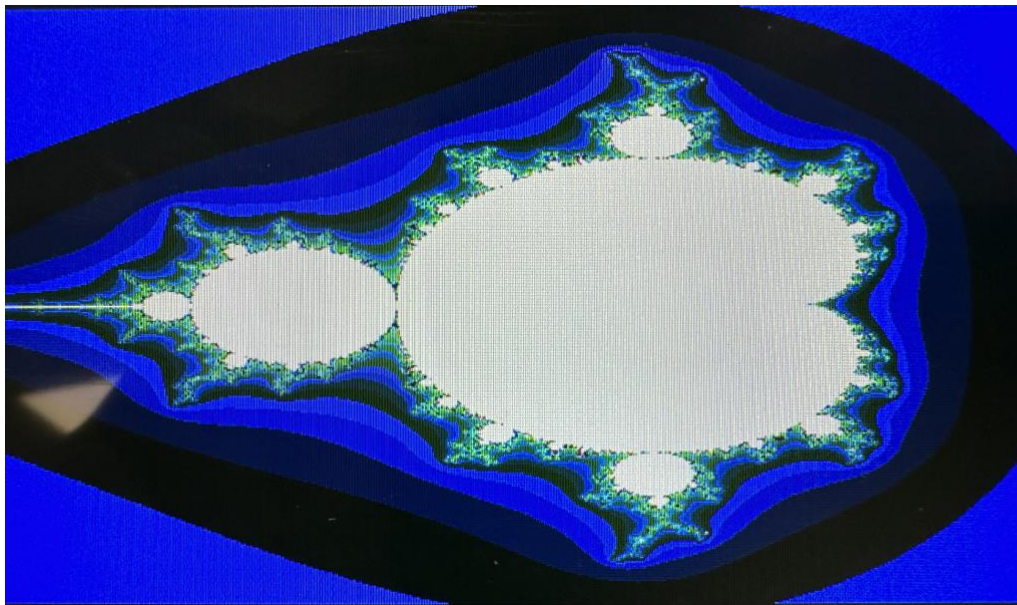


Figure 22

One processing element on the network was designated as the ‘controller’ and the remaining processing elements were designated as ‘workers’. The showcase program began by the controller resetting the performance counters then issuing a small section of the screen for each worker node to calculate the corresponding pixel colours. When a worker was finished calculating that section, it sent a packet to the controller, which responded by issuing a new section of the screen for the worker to compute. When the controller was not issuing sections to workers, it calculated a small section of the Mandelbrot set. When the entire Mandelbrot set had been computed, the controller read the values in the performance counters and sent the values over the serial connection to the host PC.

6.2 System Performance Results

The network was parametrized so that the core type could be specified in the top-level module to change the core used for synthesis. The showcase program was compiled for both the Arm Cortex-M0 and RISC-V cores using the highest optimization level (O3). Each configuration was configured to use a 50MHz clock.

6.2.1 PicoRV32 Variation Results

The PicoRV32 uses parameters that allow certain hardware features such as a hardware multiplier or barrel shifter to be instantiated in the design. The results for the PicoRV32 are subdivided into 5 subvariants each with different parameters enabled as detailed in Appendix E. The parameters found to result in the best performance when tested in the showcase program are enabled in the PicoRV32 variant labelled the ‘Selected Version’.

The network was synthesised with the compressed instruction set parameter of the PicoRV32 enabled, however the maximum clock speed of the system was less than 50MHz clock on the FPGA. As a result, the compressed instruction parameter of the PicoRV32 was not included in the comparisons.

The differences in synthesis time, maximum frequency and resource usage of the network were within 5-10% when configured for each different core. Therefore, the detailed results are included in Appendix E and only the showcase performance results are highlighted here.

Figure 23 details the number of cycles required to complete the showcase program for each variant of the PicoRV32. The ‘Multiply’, ‘Fast Multiply’ and ‘Two Stage Shift’ parameters decrease the number of cycles taken by 20-30% each. The ‘Selected Version’ with the fast multiply and barrel shifter parameters enabled completes the program in 40% of the cycles taken by the ‘Base’ version. The ‘Two Stage Shift’ option shows a significant improvement in the results as shifts are used extensively in the showcase program.

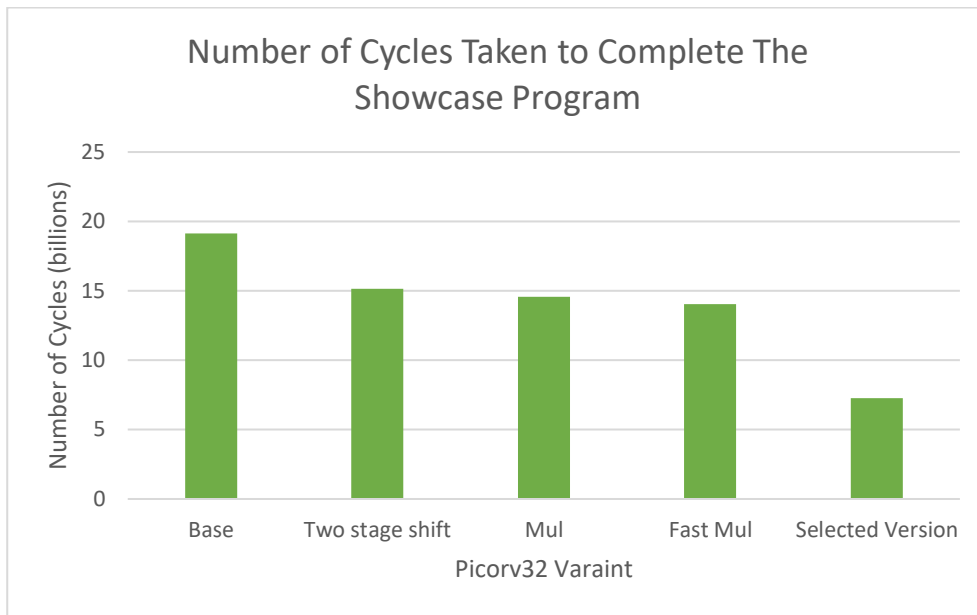


Figure 23

The number of read and writes to RAM performed during the showcase program for each PicoRV32 variant is displayed in Figure 24. The total number of writes performed during the program is constant between all the core variations. However, the number of reads is significantly fewer for the core variants using dedicated multiply hardware. This is since these cores can use the RISC-V multiply instruction set extension which replaces many shift and add instructions with a single dedicated multiply instruction, increasing code density and leading to fewer cycles spent reading instructions.

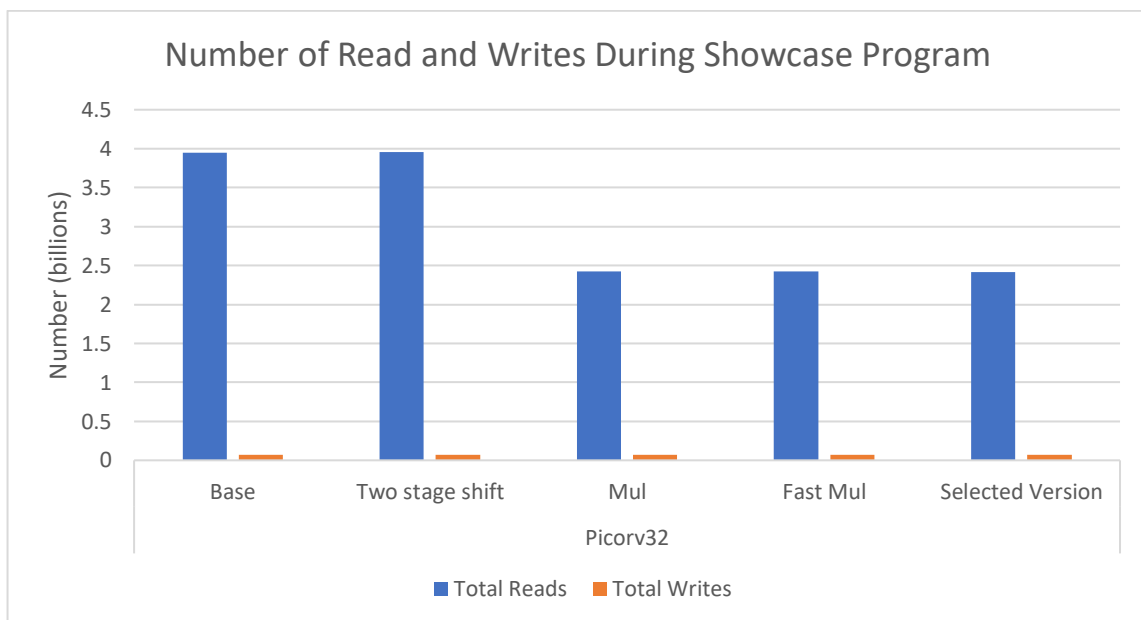


Figure 24

The ‘Selected Version’ was chosen to be compared to the other cores as it took by far the fewest number of cycles to complete the showcase program despite having a similar synthesis time and resource usage level to the other versions.

6.2.2 Core Comparison

6.2.2.1 Resource Usage

Figure 25 shows the percentage of resources used by the network when configured to use the M0, PicoRV32 and SCR1 cores.

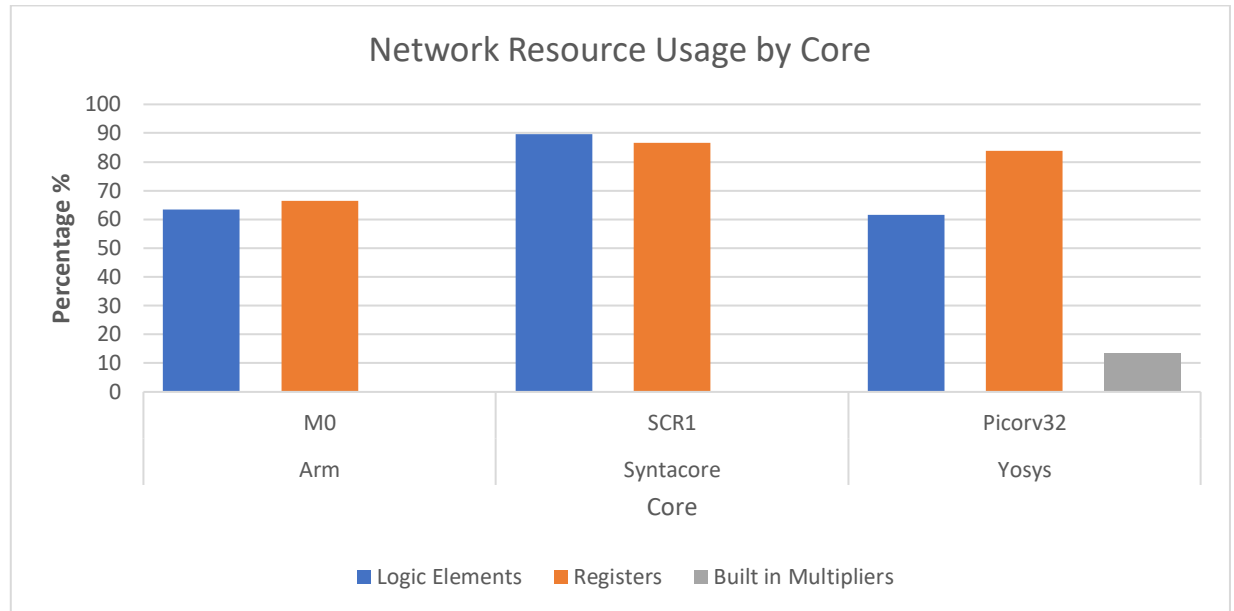


Figure 25

The SCR1 core uses approximately 40-50% more logic elements than the PicoRV32 and M0. The PicoRV32 and the M0 use approximately the same number of logic elements when synthesised. Interestingly, the PicoRV32 and the SCR1 use 30% more registers than the M0.

6.2.2.2 Synthesis Time

The time taken to synthesise the network using different cores is shown in Figure 26. The SCR1 takes by far the longest amount to synthesise, at approximately 36 minutes, which is 45% longer than the M0 and 60% longer than the PicoRV32. Some variations of the SCR1 were not able to be used as their addition would cause the network synthesis time to take longer than one hour. The M0 takes approximately 13.5 % more time to synthesis compared to PicoRV32.

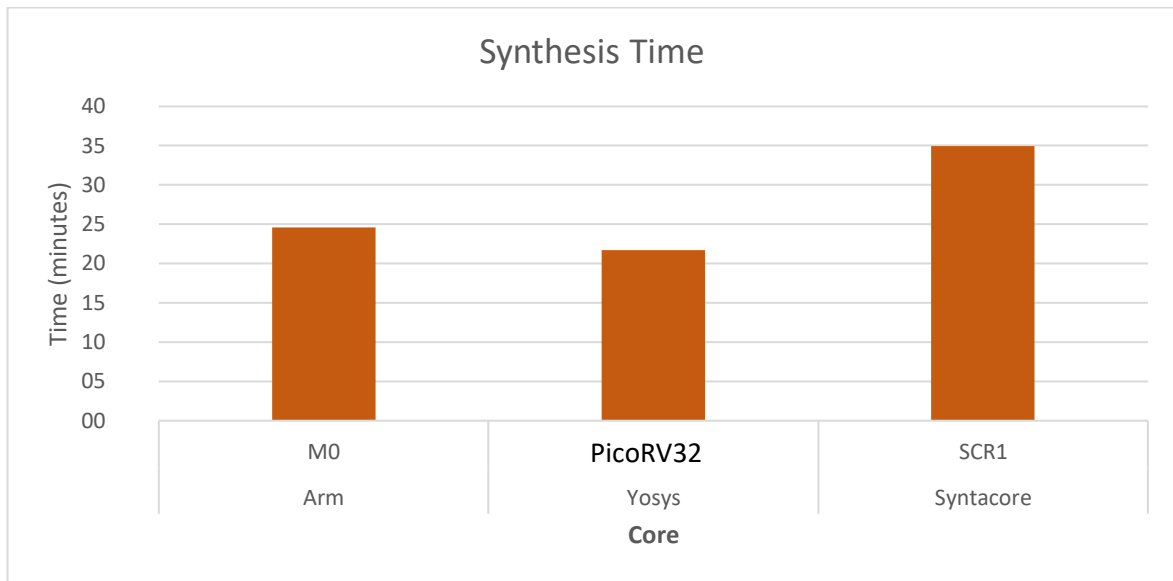


Figure 26

6.2.2.3 Maximum Frequency

Figure 27 shows the maximum clock speed of the network configured with each core after synthesis using Quartus Prime software. The SCR1 has the lowest maximum frequency out of all the cores at approximately 40Mhz. Note that this is a lower estimate for the maximum frequency, therefore the SCR1 was able to operate using the 50MHz clock during testing.

The network has a maximum clock speed of 87Mhz using the PicoRV32, which is 118% greater than the SCR1 and 53% greater than the M0.

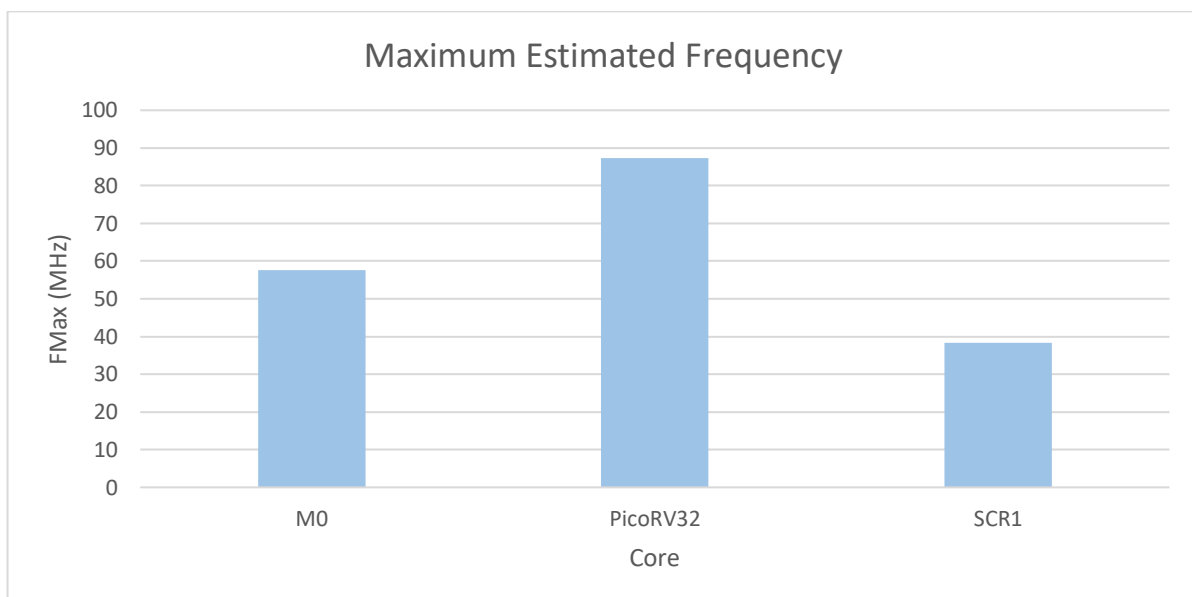


Figure 27

6.2.2.4 Compilation Time

The showcase program was compiled using the Arm and RISC-V cross compilers on the cadteaching7 server. The time taken was measured and is displayed in Figure 28. The RISC-V cross compiler was able to compile the program in considerably less time, however it can be noted that this only results in approximately 0.5 second less time to compile due to the simplicity of the program.

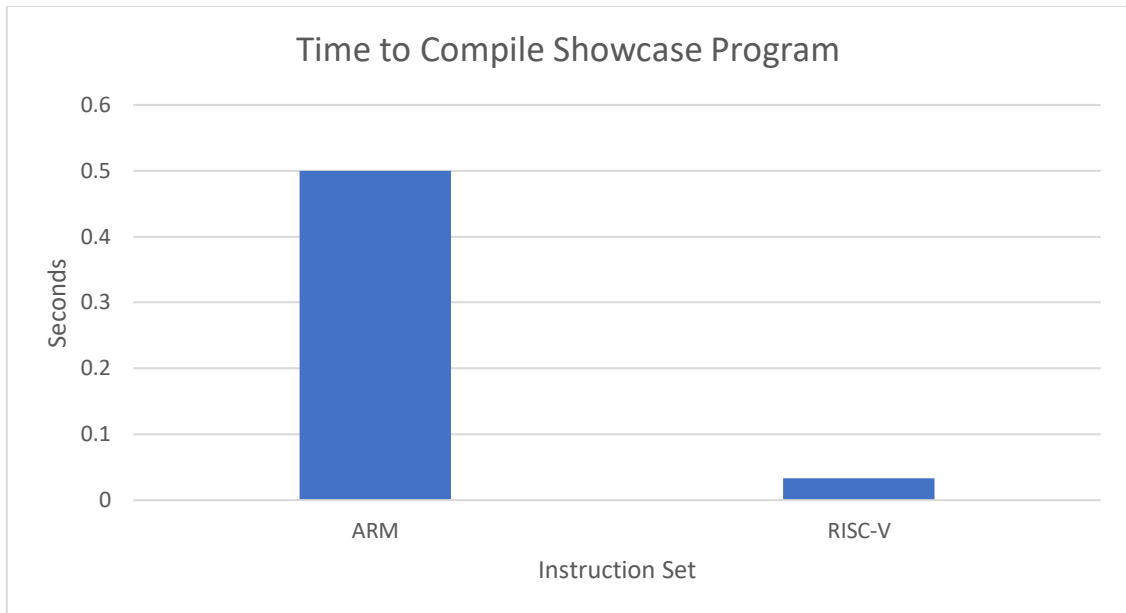


Figure 28

6.2.2.5 Simulation Time

The time taken to simulate a simple recursive multiply program taking 300,000 cycles using the XCelium simulator was recorded for the M0 and PicoRV32 cores as shown in Figure 29. The PicoRV32 takes approximately half the time of the M0 to simulate.

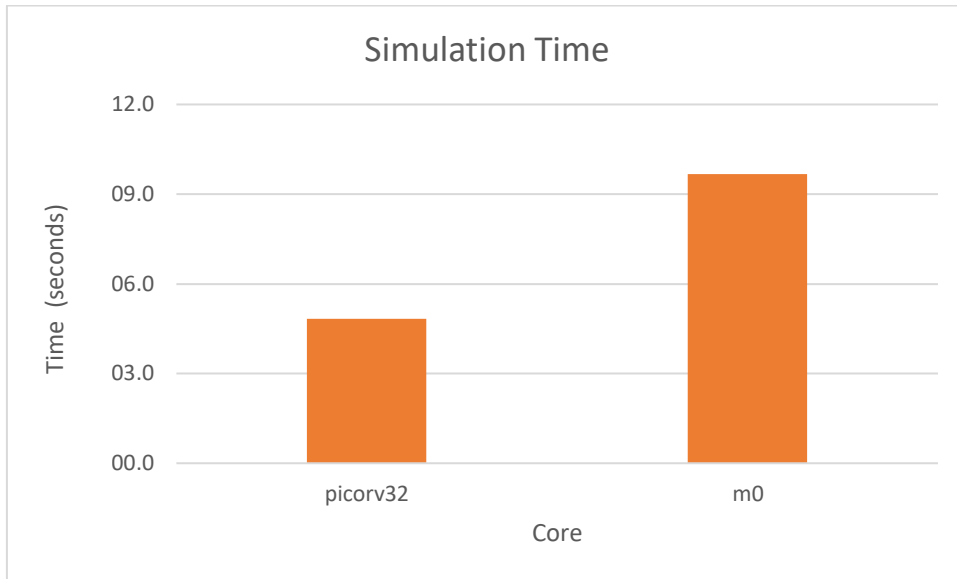


Figure 29

6.2.2.6 Showcase Program Results

Figure 30 presents the number of cycles for the network configured with each core to complete the showcase program. This test was performed using eight worker elements without the controller calculating sections of the Mandelbrot set. The M0 completes the showcase program the fastest, in approximately 4.5 billion cycles. The SCR1 completes the program in the second fewest number of cycles, taking 50% more cycles than the M0. The PicoRV32 performed considerably worse; requiring 80% more cycles to complete the showcase program compared to the M0.

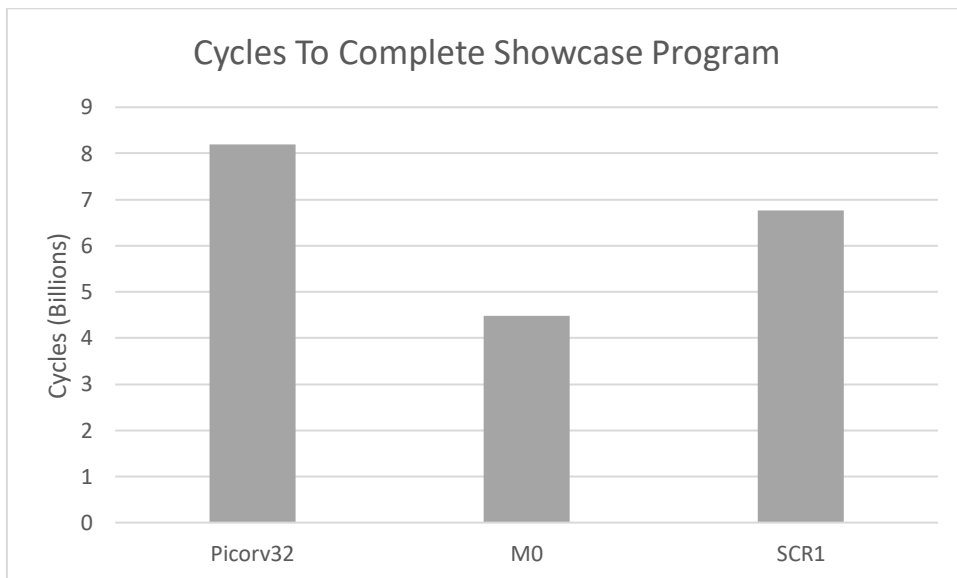


Figure 30

The time taken to complete the showcase program can be calculated by multiplying the number of cycles taken by the period of the 50MHz clock used in the test. The time taken by the cores to complete the program is shown in Figure 31 alongside a predicted time shown in red for the PicoRV32 and M0 cores operating at their maximum clock frequencies. When taking into account the maximum frequencies of the cores, the extra time taken by the PicoRV32 compared to the M0 is reduced to approximately 20%.

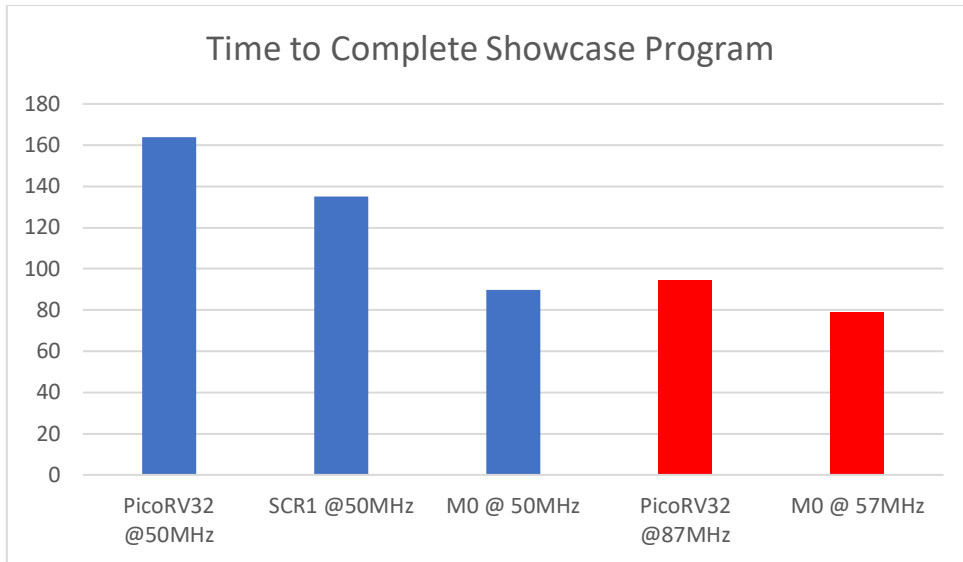


Figure 31

6.2.3 Network Performance

The performance of the network was evaluated by gathering performance metrics from the 'Selected Version' PicoRV32 variant. The time taken for the showcase program to be completed was calculated by sending the total cycle count from the controller processing element to the host PC. Figure 32 displays the time taken for the showcase program to be completed as the number of processing elements was increased. The number of processing elements used to calculate the showcase program includes the controller processing element as described in section 6.1.

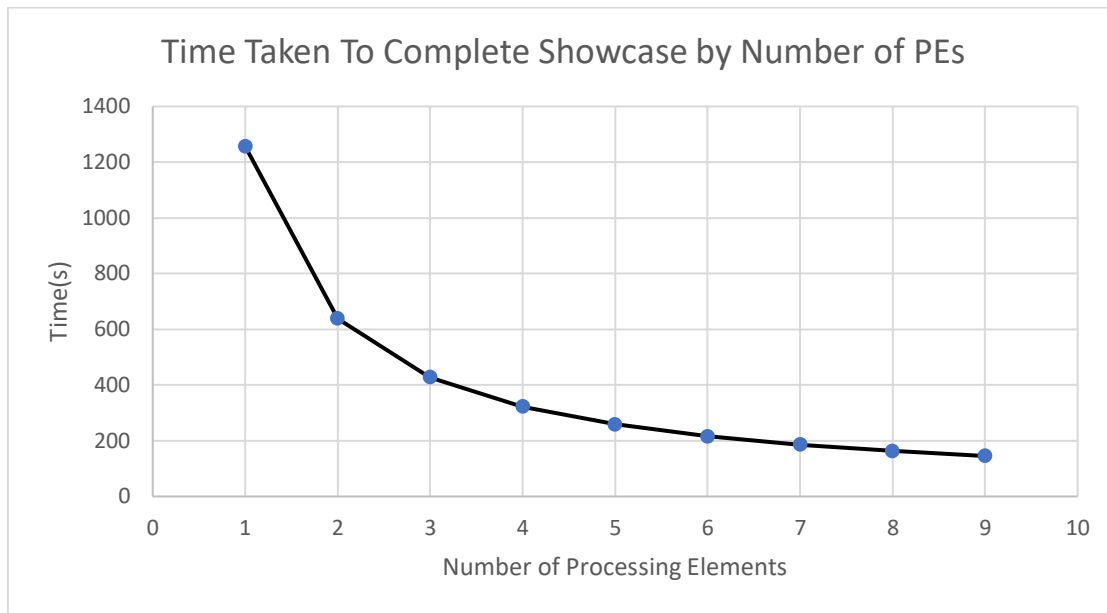


Figure 32

The relative speedup in the time taken to complete the showcase program for each added processing element is shown in Figure 33. There is a linear relationship between the speed up of the showcase program against the number of processing elements. The speedup is averaged to be approximately 95.8% per processing element added to the network.

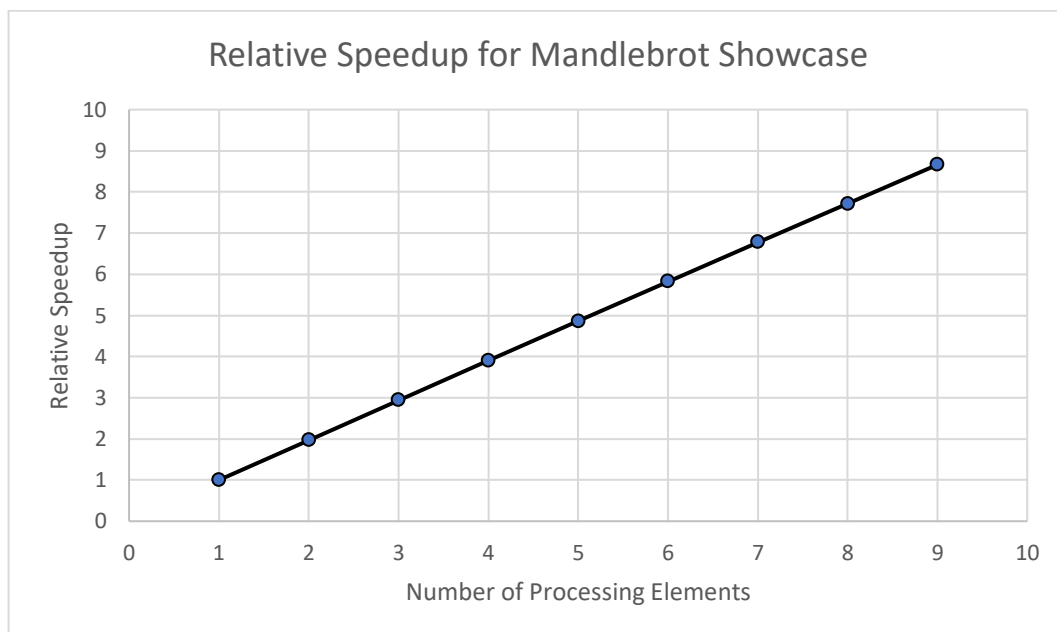


Figure 33

6.3 Evaluation

The linear speed up in the time taken to complete the showcase program by using the network suggests that the communication overhead by using the network does not cause a major slowdown in the execution of the program at this level of network traffic.

The M0 core uses fewer cycles to complete the showcase program despite using approximately 30% fewer logic elements and taking 40% less time to synthesise than the PicoRV32. In addition, the M0 outperforms the SCR1 in every measured metric, therefore the SCR1 is no longer considered. Compared to the PicoRV32, the M0 provides a much better cycle performance, taking 54% fewer cycles, however the M0 only takes 20% less time to complete the showcase program when adjusting for the high maximum frequency for the PicoRV32. The results suggest that the Arm Cortex-M0 has a superior performance compared to the RISC-V cores when tasked to complete multiplication heavy programs.

However, the PicoRV32 has a similar performance and may be more suited to applications that do not require many multiplications. Instead, it could benefit an application where an optional parameter such as a two-stage shifter could be utilized and a hardware multiplier would be wasted resource.

Another benefit of the PicoRV32 is the fact that the high maximum frequency of the PicoRV32 allows it to have lots of timing slack when running at a lower frequency. This allows it to be integrated into other designs “without compromising timing closure” [2]. In addition, the core does not require a licence fee to use, therefore it could be suited to be used as an auxiliary processor in a larger system on chip.

The heavily reduced simulation time and moderately decreased synthesis time make the PicoRV32 a good choice for university projects with limited schedules as it can be debugged faster.

7 Project Management

At the early stages of the project, work progressed faster than expected, allowing more time to be used to add features not specified in the project brief such as the bootloader system. However, long delays were caused by issues debugging these added features, leading to less time being available to investigate the performance of the different cores.

The proposed project schedule made in Week 3 of the project and the actual recorded schedule are presented in the Gantt charts in Appendix A.

The risk assessment in Appendix B highlights that the largest risk in this project was an unrealistic project schedule and scope. For this reason, a generous amount of contingency time was added to the schedule. This proved useful as four out of the six weeks of planned contingency time were used in the project. Two weeks were used to ensure progress on the network design and two weeks was used to gather results and write the final report.

Communication over the network was designed and tested two weeks ahead of schedule due to the mentioned use of contingency time. This freed up project time to be committed to the development of the bootloader subsystem that was identified to be an important additional feature early in the project.

Bad Verilog coding techniques used at the start of the project led to un-synthesisable RTL that slowed down progress. Five weeks were spent debugging the serial link and bootloader module due to differences between simulation and synthesis behaviour caused by un-synthesisable RTL.

A large amount of project time was dedicated to integrating existing publicly available RISC-V cores into the design. Many issues were encountered due to the cores not being compatible with the Cadence XCelium simulator and difficulties in including necessary design files for simulation.

The amount of project time scheduled to the performance evaluation was inadequate as it took almost eight weeks to complete compared to the planned two weeks. This is in part due to the extra time taken to integrate the SCR1 core which was not considered when first planning the project.

Throughout the project, the design has been version controlled using Git, with each commit documented and named to reflect updates to the design. A total number of 306 commits were logged. Using Git to version control the project made debugging substantially easier as changes between successive commits could be analysed line by line using the GitHub blame tool.

An off-chip SRAM memory chip was intended to be used as a framebuffer but required clock domain crossing logic which was unable to be implemented in the time available. In addition, the interrupt-based message passer was only implemented for the Arm Cortex-M0 and not the RISC-V cores due to differences in the RISC-V and Arm interrupt processes.

The serial link connection reduced the time taken to gather performance results as tests could be run on the network itself and the results communicated to the host PC instead of running lengthy simulations. In addition, the bootloader subsystem decreased software development time as the showcase program could be modified and tested on the network without resynthesizing the system.

All design decisions and problems encountered were written in a logbook, which was useful as a reference for resolving repeated problems and for writing this report. A total of two logbooks were used throughout the project.

8 Conclusion

The aim of this project was to build a parallel computing network of RISC-V processors and to compare the performance of these cores in the place of the Arm Cortex-M0 used in previous projects. A 2D mesh network was built and then programmed to run a showcase program using a bootloader system. The results of the showcase program were sent to a host PC using a serial link. The test demonstrates that the network is capable to facilitate communication between cores to enable a linear speedup in the time taken to complete a simple parallel program.

The open-source Yosys PicoRV32 and Syntacore SCR1 RISC-V CPUs were selected to be compared to the Arm Cortex-M0. Overall, the Arm Cortex-M0 was evaluated to have the best practical performance as it uses approximately the same number of resources as the PicoRV32 and is predicted to complete the specific showcase program in 20% less time than the PicoRV32 operating at maximum clock speed. In addition, the M0 was found to also be faster than the SCR1 core despite using far fewer resources and taking half the time to synthesise. However, the PicoRV32 has the significant advantage of taking only half the time to simulate compared to the Arm Cortex-M0, making it useful for university projects.

Most importantly, the RISC-V cores can be used and customized for any purpose without a license. The disadvantages of these cores, such as reduced performance for the PicoRV32 and increased resource usage by the SCR1, may be worth it in certain applications when considering they come with no restrictions.

8.1 Future Work

My recommendations for future research will be to investigate further the performance of PicoRV32 compared to the Cortex-M0 using more realistic test programs. Testing the performance and power usage of the PicoRV32 at its maximum frequency will be useful in a fair comparison between it and Cortex-M0. In addition, if the PicoRV32 can be modified to support pipelined memory access, it will be useful to see if this improves performance.

Further research could also be spent investigating a wider range of RISC-V cores available online to determine if a RISC-V core can achieve comparable performance to the Arm Cortex-M0.

An interrupt-based showcase program was not implemented for the RISC-V cores, therefore the difference this feature could have made to their performance was not investigated. It would be interesting to evaluate how the differences between the RISC-V and Arm instruction sets interrupt handling processes affects their performance in a suitable test.

The main body of this report is 8589 words long, counted using Microsoft word.

9 References

- [1] I. McNally, “ARM System on Chip Altera FPGA example,” [Online]. Available: https://secure.ecs.soton.ac.uk/notes/bim/notes/cad/lab/system_on_chip/.
- [2] C. Wolf, picorv32, GitHub, 2022.
- [3] Syntacore, [Online]. Available: <https://syntacore.com/page/products/processor-ip/scr1>. [Accessed 25/06/2023].
- [4] R.-V. International, “<https://riscv.org/about/>,” 2021. [Online].
- [5] D. A. a. H. Patterson, Computer Organization and Design RISC-V Edition: The Hardware Software Interface, 2017, pp. 490-553.
- [6] D. Sima, “Advanced computer architectures : a design space approach,” pp. 587-611.
- [7] M. Vousden, “ELEC3219 Advanced Computer Architecture Interconnects,” 2023-03-02. [Online].
- [8] “CS838: Topics in parallel computing,” University of Wisconsin-Madison. [Online].
- [9] C. J. a. N. L. M. Glass, The Turn Model for Adaptive Routing, 1992.
- [10] C. E. Cummings, “Simulation and Synthesis Techniques for Asynchronous FIFO Design”.
- [11] “Cortex-M0,” [Online]. Available: <https://developer.arm.com/Processors/Cortex-M0>.
- [12] “AMBA AHB Protocol Specification,” [Online]. Available: <https://developer.arm.com/documentation/ih0033/latest/>.
- [13] I. McNally, “Altera System On Programmable Chip,” 2021. [Online].
- [14] Y. L. D. P. K. A. Andrew Waterman, “The RISC-V Compressed Instruction Set Manual,” 2015.
- [15] M. G. L. Eric Peña, “UART: A Hardware Communication Protocol Understanding Universal Asynchronous Receiver/Transmitter,” [Online].
- [16] “Plotting algorithms for the Mandelbrot set,” [Online]. Available: https://en.wikipedia.org/wiki/Plotting_algorithms_for_the_Mandelbrot_set.
- [17] L. Y. Fai, “MIMD Parallel Computer Design using Multiple FPGA Development Boards,” 2020.
- [18] A. Siabi, “MIMD Parallel Processing with Distributed Memory on Multiple FPGAs,” 2020.
- [19] L. Woolmer, “MIMD Hypercube Multi-Processing Network with Distributed Memory and Low-Latency Message-Passing,” 2021.
- [20] J. Tait, “Design and evaluation of a MIMD machine with virtual shared memory built on message passing with virtual cut-through,” 2018.
- [21] J. Hamblen, RAZZLE, Available at: <http://hamblen.ece.gatech.edu/UP3/>.
- [22] C. J. G. a. L. M. Ni, “The Turn Model for Adaptive Routing”.
- [23] “ARM Ltd. design start IP,” [Online]. Available: <https://www.arm.com/resources/designstart>.

10 Appendix A - Project Schedule

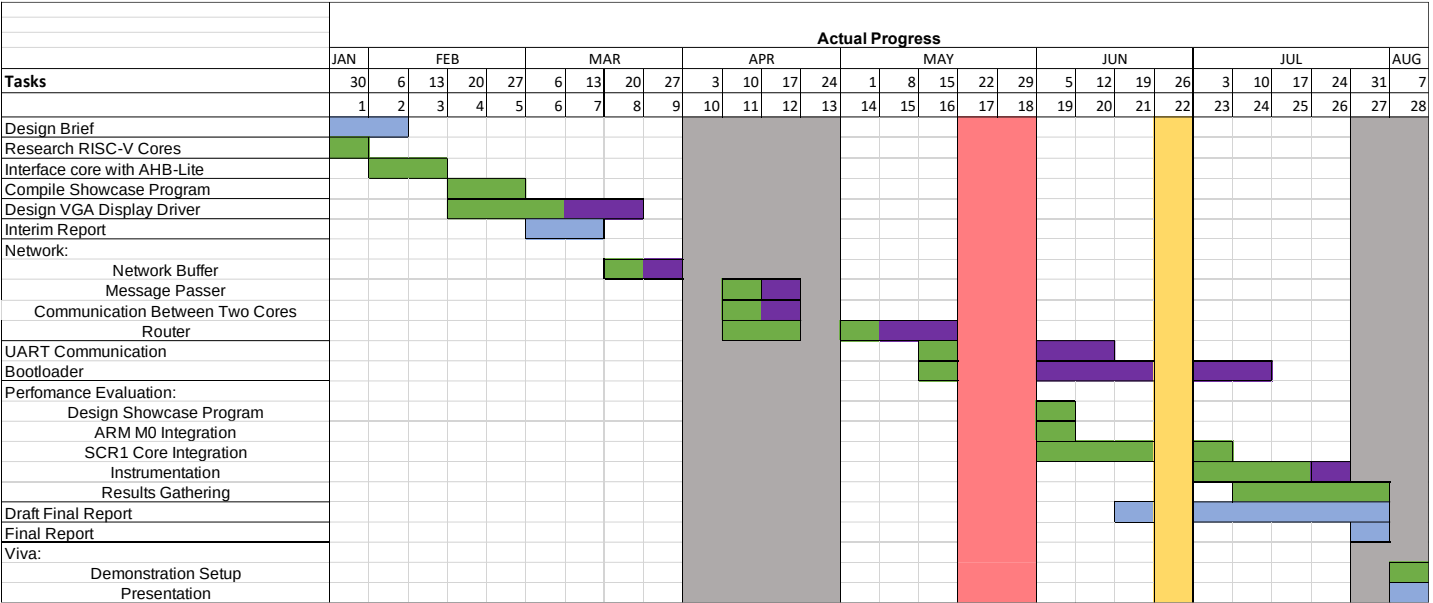


Figure 34 Recored Progress

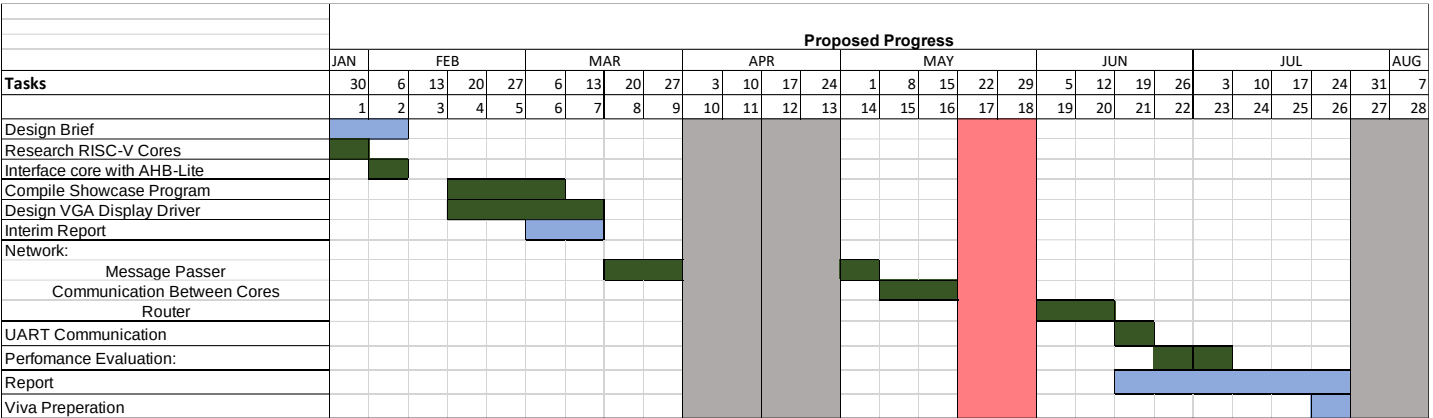


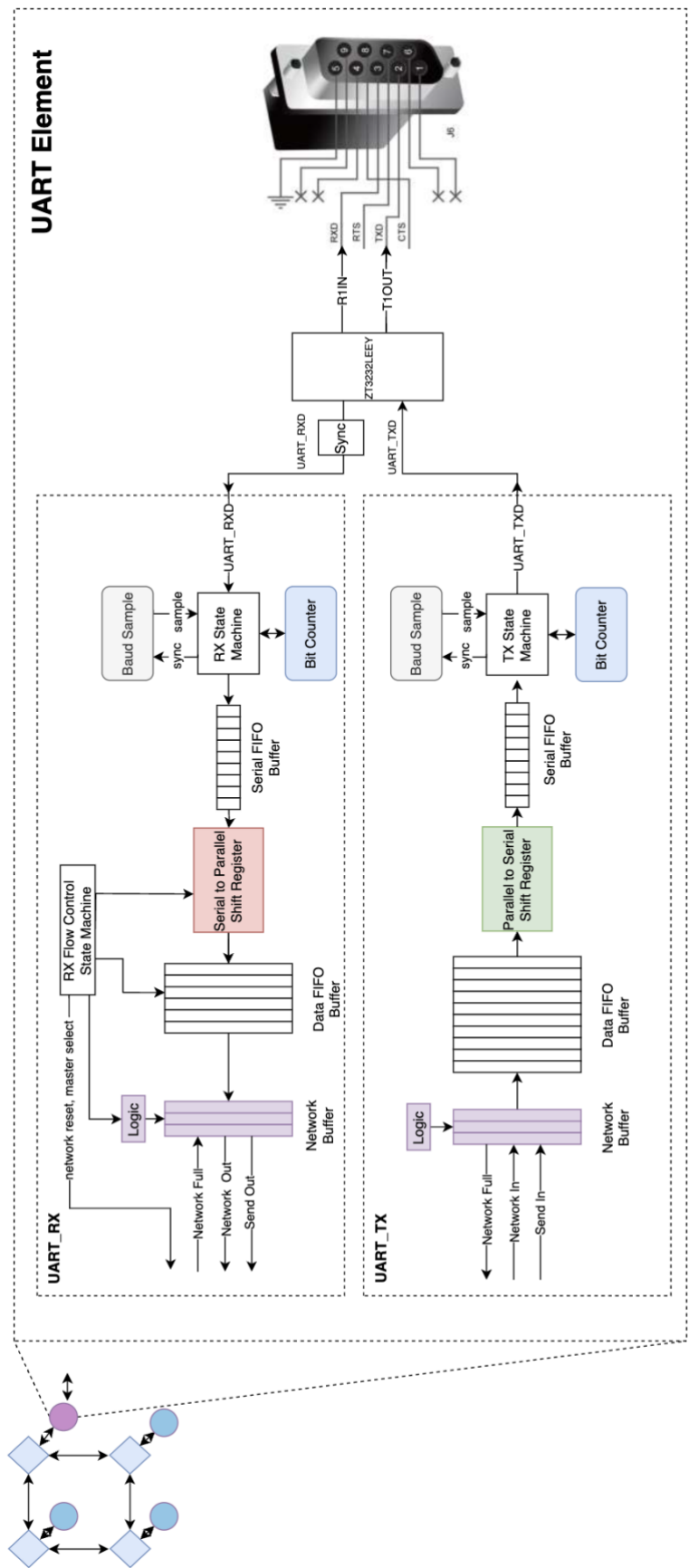
Figure 35 Scheduled Progress

	Design
	Testing
	Proposed Design and Testing Time
	Documentation
	Exams
	Contingency
	Break

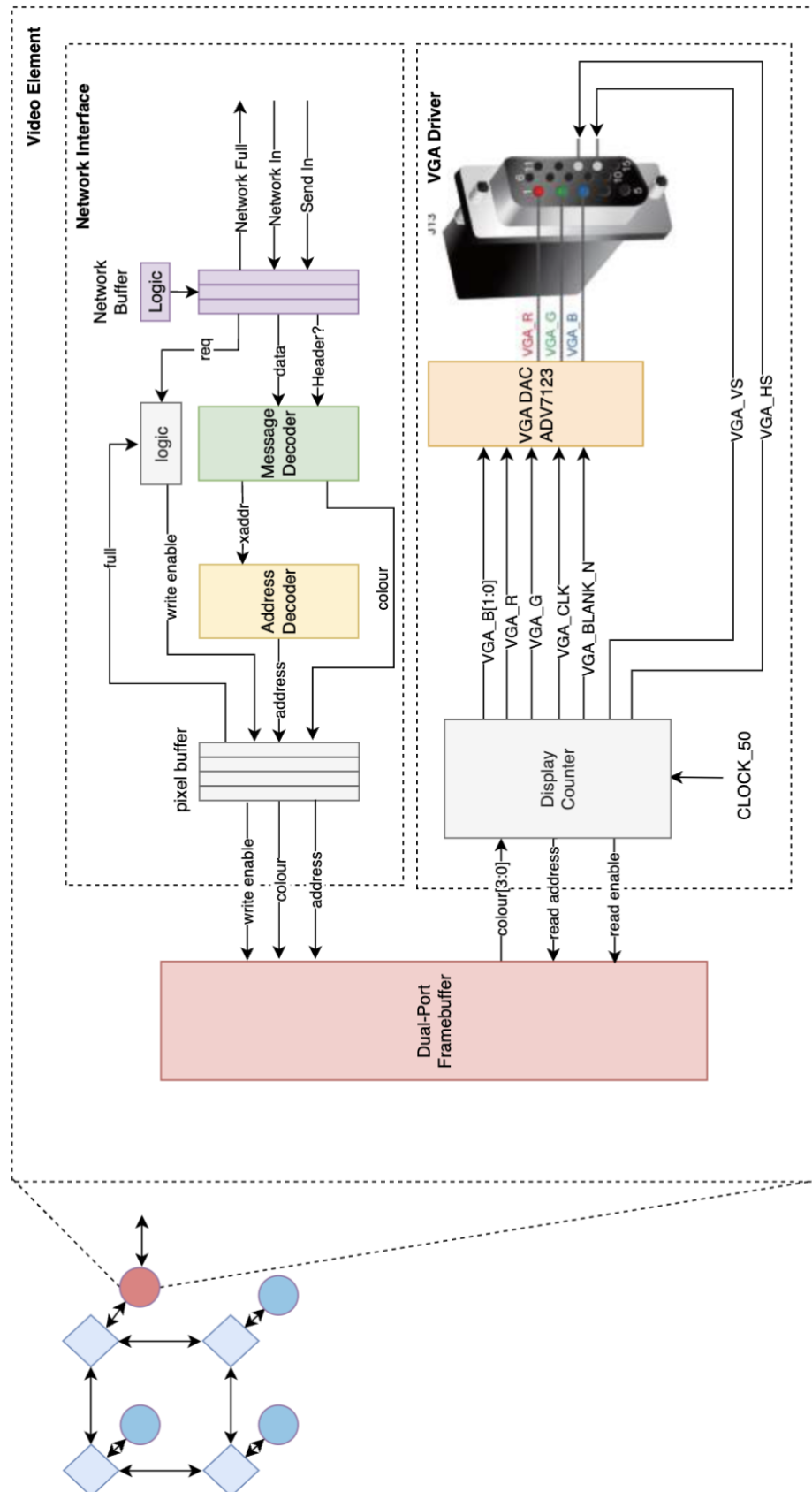
11 Appendix B – Risk Assessment

Problem	Loss /5	Prob /5	Risk /5	Plan	Controlled Loss /5	Controlled Probability /5	Controlled Risk
Loss of RISC-V core licence	4	1	4	Find several RISC-V core candidates and choose a core that uses a similar memory interface	2	1	2
Illness preventing progress	2	3	6	Incorporate contingency time into project timeline	1	3	3
Loss of project files or report due to computer failure	4	2	8	Backup project files to local PC regularly and use Git for version control	3	1	3
Loss of documentation due to computer failure	4	2	8	Use onedrive autosave feature	3	1	3
Unrealistic deliverable deadlines	3	4	12	Incorporate contingency time into project timeline and focus on core deliverables	2	3	6

12 Appendix C – Serial Link Node Diagram

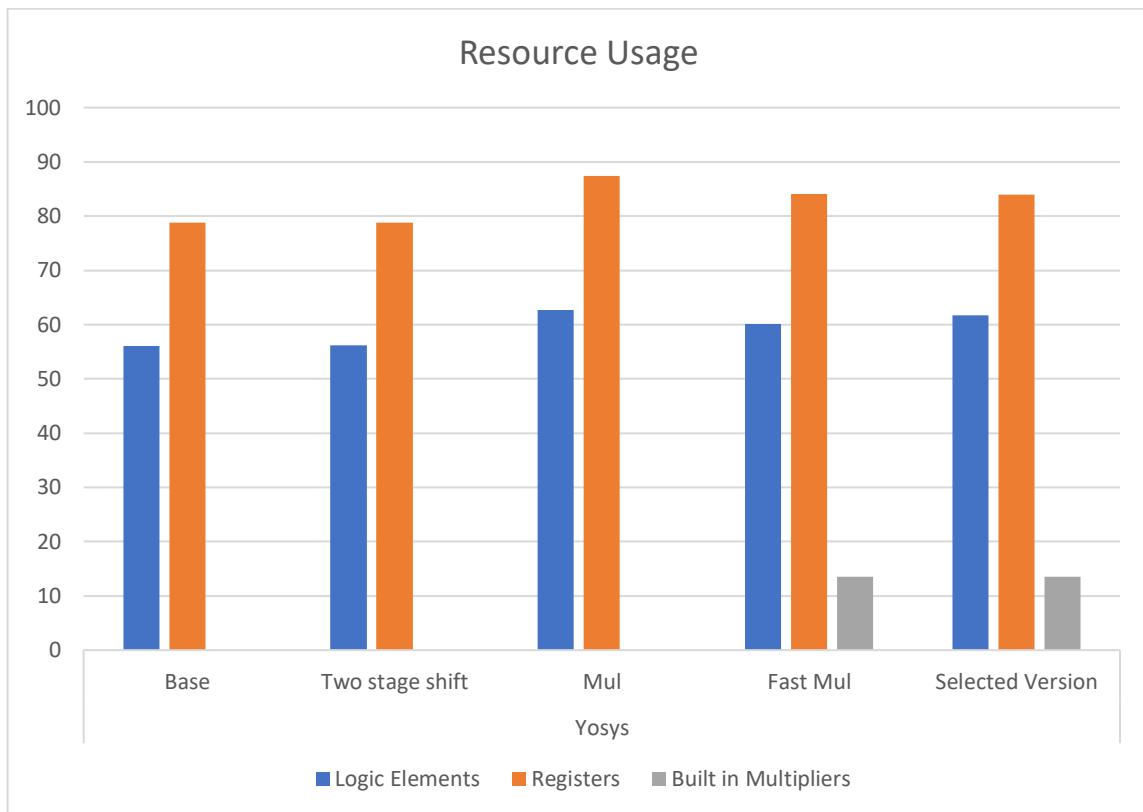


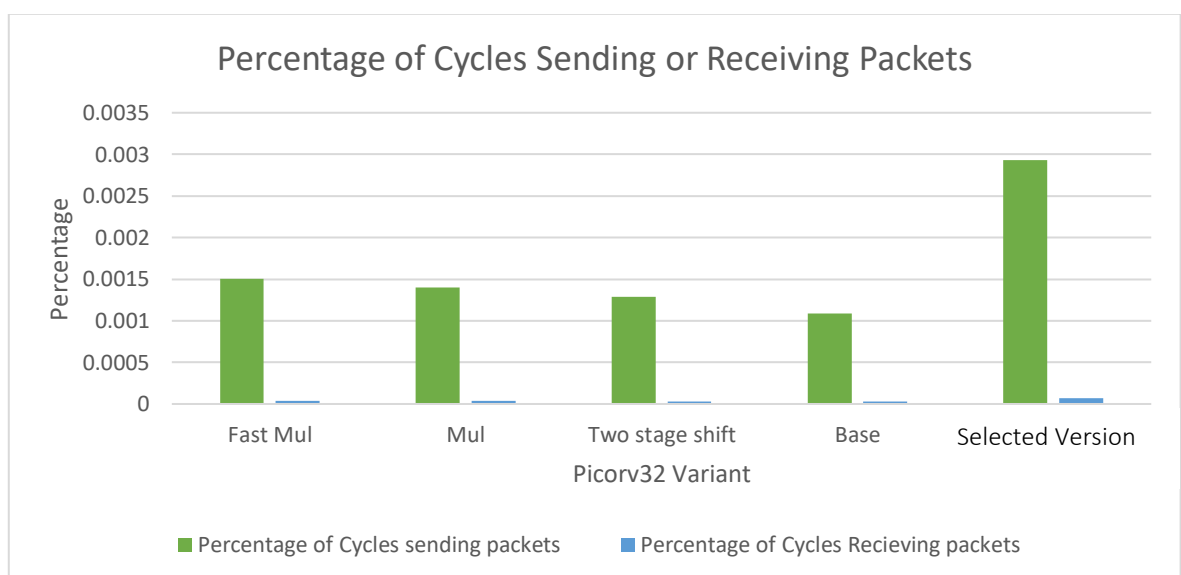
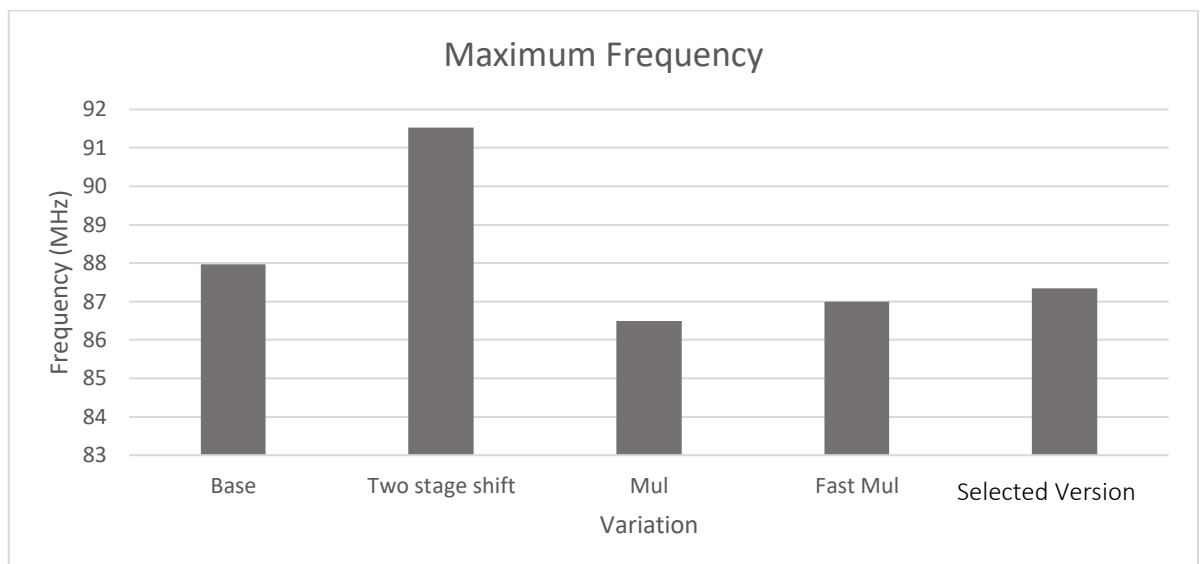
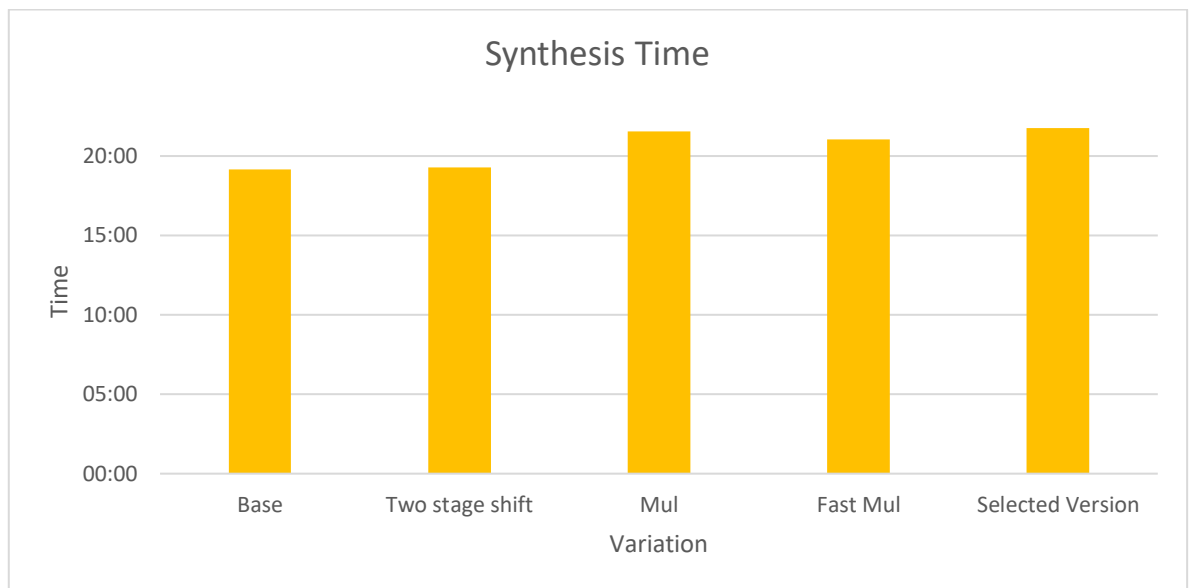
13 Appendix D – Video Node Diagram



14 Appendix E – Performance by PicoRV32 Variation

Picorv32 config	RV32IM	RV32IM	RV32I	RV32IM	RV32I
	Fast Mul	Mul	Base	High Performance	Two stage Shift
Mul	1	1	0	1	0
Fast Mul	1	0	0	1	0
Barrel Shifter	0	0	0	1	0
Division	1	1	0	1	0
Compressed Extension	0	0	0	0	0
Two Cycle ALU	0	0	0	0	0
Two Stage Shift	0	0	0	1	1





15 Appendix F – Results Data

Memory Accesses for Showcase program using 9 PEs - Picorv32

		Total Cycle Time	Total Reads	Total Writes	Total Memory Accesses
picorv32	Fast Mul	14045039677	2423930394	71092314	2495022708
	Mul	14583887498	2424955855	71076734	2496032589
	Two stage shift	15169366247	3959491338	71243272	4030734610
	Base	19113905987	3946099326	71139451	4017238777
	Selected Version	7254338079	2415684961	70873026	2486557987

Cycles sending and Receiving Packets showcase program 9 PEs - Picorv32

		Cycles Sending Packets	Cycles Reading Packets	Total Cycles
picorv32	Fast Mul	211273	4932	14045039677
	Mul	203932	4988	14583887498
	Two stage shift	195220	5050	15169366247
	Base	207559	4956	19113905987
	Selected Version	212305	4920	7254338079

Synthesis Summary

		Logic Elements	Registers	Built in Multipliers used	Synthesis Time
Arm	M0	72652	33183	0	24:37
Picorv32	Base	64186	39394	0	19:09
	Two stage shift	64365	39394	0	19:17
	Mul	71,742	43705	0	21:34
	Fast Mul	68784	42013	72	21:04
	High Performance	70614	41950	72	21:45
SCR1	Mul	102,625	43283	0	

Fmax		
	Core	Fmax(MHz)
Arm	M0	57.53
Picorv32	Base	87.97
	Two stage shift	91.52
	Mul	86.5
	Fast Mul	86.99
	High Performance	87.35
Yosys	Selected Version	87.35
SCR	Mul	38.33

Picorv32 Showcase Time by Number of Cores

Number of PEs	Time (seconds)
1	1256.54605
2	638.358404
3	427.46924
4	321.798148
5	258.553659
6	215.661497
7	185.324779
8	162.89552
9	145.086762

Cycles to complete Showcase Program using 8 workers

Core	Cycles
Picorv32	8199610026
M0	4483468733
SCR1	6759101981

Time to complete Showcase Program using 8 workers

Core	Time (Seconds)
PicoRV32 @87MHz (Predicted)	94.2483911
PicoRV32 @50MHz	163.9922005
M0 @ 57MHz (Predicted)	78.65734619
M0 @ 50MHz	89.66937466